PRODUCT SPECIFICATION / ENGINEERING PLAN

Corral

통제된 자율성,
실시간으로 조율하는 AI 팀.

Controlled autonomy. Visible collaboration.
Deliberate steering.

D&D 하네스 설계 · LLM 에이전트 협업 · 실행 관찰
Reins 스티어링 · SDD / TDD 구현 체계

48	50	112
요구사항	구현 태스크	계획 테스트 시나리오

Bell / 개인 프로젝트
v0.1 · 2026-10-14
구현 착수용 설계 패키지. 앱 구현·라이브러리 통합·제품 테스트 완료 보고서가 아닙니다.

READING MAP

문서 구성

제품의 책임과 완료 기준을 먼저 고정하고, 구현 계약과 테스트로 내려갑니다.
편집 가능한 Markdown·Mermaid·JSON 및 템플릿은 ZIP에 함께 제공합니다.

01 **제품·실행 스펙** 3
목표 · 이름과 단위 · D&D · 협업 · Reins · 수용 기준

02 **기술 설계** 17
상태 소유 · Coordinator · GraphRunner · Pi · 저장 · 이벤트

03 **API·이벤트 계약** 32
명령 · Receipt · 오류 · SSE · 이벤트 payload

04 **세부 구현 플랜** 36
SDD / TDD · P0-P9 · 50개 태스크 · 일정 · 완료 게이트

05 **테스트 전략** 46
불변식 · 경합 · Mock과 실제 LLM 평가 · 성능 · 보안

06 **아키텍처 결정 기록** 50
13개 ADR · 선택 근거 · 대안 · 제약

07 **라이브러리·출처** 53
OSS 선정 · 유지보수 관측 · 버전 잠금 · 원자료

08 **문서 검증과 실행 준비** 61
검증 범위 · 생성 파일 · 아직 검증하지 않은 부분

첫 MVP의 필수 조건
D&D + 협업 + Loop + subagents + Reins.
실행 중 그래프 구조 변경과 crash resume은 포함하지 않습니다.

PART 01

제품·실행 스펙

목표 · 이름과 단위 · D&D · 협업 · Reins · 수용 기준

> **Controlled autonomy. Visible collaboration. Deliberate steering.**
> D&D로 AI 팀을 설계하고, 규칙 안에서 자율 협업하게 하며, 진행 중에도 적용 범위와 시점을 확인하면서 개입하는 로컬 우선 도구.

문서 관리	값
기준일	2026-10-14
상태	구현 착수용 설계안. 제품 기능 구현·통합 검증은 아직 수행하지 않음
소유자	Bell
개발 방식	SDD(Spec-Driven Development) + TDD(Test-Driven Development)
기준 자료	리뷰어 MVP v1.1 PDF [B1] + 대화에서 확정한 D&D·포트폴리오·실시간 개입 요구
관련 문서	구현 플랜, 기술 설계, 의존성과 출처, ADR

0. 이 문서의 결정 권한과 읽는 순서

사용자가 확정한 목표 → 본 제품 스펙 → 기능별 수용 기준 → 기술 계약 → 구현 태스크 순서로 해석한다. 라이브러리 편의 때문에 D&D·협업·스티어링을 제외해서는 안 된다. 테스트가 실패한다는 이유로 명세를 묵시적으로 약화하지 않는다.

[B1]의 범위·구조·용어를 출발점으로 삼되, 아래의 보완은 **이번 문서에서 새로 내린 설계 결정**이다. 외부 프로젝트가 이미 이 통합 기능을 제공한다는 주장이 아니다. 라이브러리 사실과 확인 날짜는 별도 출처 문서에 분리했다.

구분	처리
그대로 유지	첫 MVP D&D, 4종 노드, 협업 메시지, 동적 자식 실행, 제한된 검토 루프, 텍스트 산출물, 전체·개별 취소, 실행 중 구조 고정 [B1 §1·3·4·7]
구체화	상태 소유자=Run Coordinator, SQLite의 영속 이벤트·투영을 한 트랜잭션에서 확정, LangGraph/Pi는 실행 어댑터
명시적 수정	일반 지시는 전체 정지 없이 접수 가능. 실제 전달은 다음 요청 경계. 강제 방향 변경은 일시정지/취소 후 처리
명시적 수정	작업 상태에 실패·취소, 실행 상태에 대기열·일시정지 중·취소 중을 추가

구분	처리
명시적 수정	모든 메시지에 산출물 버전을 요구하지 않고, 질문·답변·제출별 필수 참조를 다르게 정의
명시적 수정	실행 전 검증을 3종에 한정하지 않고, 지원 명세의 스키마·참조·포트·권한 검증까지 포함
명시적 수정	산출물 인과를 생성 시각이 아닌 요청 스냅샷·입력 버전으로 연결
유지하되 후속 일정 명시	Local/Jev는 MVP 직후 v0.2의 독립 마일스톤. 자동 최적화·Hybrid 정책 평가와 분리

1. 제품 목표와 사용자

1.1 첫 MVP가 증명할 것

사용자가 **코드나 YAML 편집 없이** 조사·작성·검토 팀을 구성한다. 조사자가 자식 에이전트를 호출하고, 작성자가 조사자에게 질문하며, 검토 반려가 다음 반복의 수정 작업으로 이어진다. 사용자는 그 도중 지시·미래 모델 설정·한도를 바꾸거나 일부 작업을 멈출 수 있다. 화면에서 각 변경의 접수·적용·결과 연결을 확인한다.

세 가지 제품 질문을 동등하게 검증한다.

1. **설계:** 내가 만든 노드·연결·역할 설정이 실제 실행을 바꾸는가?
2. **협업:** 작업·질문·자식 실행·산출물의 연결을 이해할 수 있는가?
3. **개입:** 바꾼 지시가 누구에게 언제 적용되는지 신뢰할 수 있는가?

1.2 대상과 환경

주 대상은 자신만의 LLM 에이전트 팀을 구성하려는 개발자·파워유저다. Bell의 프런트엔드 포트폴리오로서 편집 UX, 실시간 상태 관리, 접근성, 성능, 오류 복구 설계를 드러낸다. 코딩 전용 제품으로 제한하지 않지만, MVP 실사용 시나리오는 조사→작성→검토 하나로 검증한다.

배포 형태: 데스크톱 브라우저 + 로컬 Node.js 서버 + SQLite. 클라우드 계정과 설치형 데스크톱 셸은 필수가 아니다. 워크스페이스당 활성 Run은 1개로 제한하되, 과거 Run은 여러 개 보관한다. 이는 이번 MVP의 자원·UI 범위 결정이다. 다수 에이전트의 병렬 작업은 한 Run 안에서 지원한다.

포트폴리오 모드: 동일한 편집기·상태 투영·Reins를 사용하되 실행 어댑터만 결정론적 mock으로 바꾼 정적 웹 데모를 제공한다. 실제 LLM처럼 가장하지 않으며 `Demo / Simulated agents`를 항상 표시한다.

1.3 성공과 비성공

성공은 단순히 최종 문서가 나오는 것이 아니다. 사용자가 화면으로 “이 버전이 왜 생겼는가”, “지금 누구를 기다리는가”, “내 지시는 전달되었는가”, “남은 추가 실행은 얼마나 허용되는가”에 답할 수 있어야 한다.

멀티에이전트가 단일 에이전트보다 항상 싸거나 정확하다는 목표는 두지 않는다. 비용·지연·품질 개선은 실제 비교 실험으로 별도 평가한다. ‘모델의 내면을 모두 보여준다’는 표현을 쓰지 않는다. 표시하는 것은 요청·응답·도구·작업·메시지·설정 등 관측 가능한 기록이다.

2. 이름·단위·UI 용어의 확정

2.1 은유는 제품 이해를 돕는 곳에만 사용한다

Corral은 제품명인 동시에, 팀·작업 그래프·도구 권한·공유 예산을 감싸는 실행 경계다. Loop와 같은 의미로 사용하지 않는다. 팀 하나가 여러 번 반복될 수 있으므로 팀의 경계와 반복 제어는 분리해야 한다.

MVP의 캔버스 전체가 하나의 Corral이다. 그 안에 Loop 영역과 Agent·Decision·Output을 배치한다. 여러 Corral을 중첩하거나 통째로 드롭하는 재사용 Subflow는 v0.3 이후로 둔다. 이렇게 하면 이름 때문에 다섯 번째 실행 노드나 새로운 중첩 런타임이 생기지 않는다.

제품/UI 용어	코드의 정식 타입·단위	의미와 경계
Corral Studio	`apps/studio`	설계·실행·협업·개입을 제공하는 웹 앱
Corral	`CorralDefinition`	하나의 팀, 그래프, 권한, 상한을 묶는 저장 가능한 정의
Workspace	`Workspace`	Corral 정의·자료·실행 이력을 보관하는 로컬 디렉터리
Canvas	`CanvasLayout`	노드 좌표·크기·선택·뷰포트. 실행 의미와 분리
Agent	`AgentNode` / `AgentRole`	노드는 배치, Role은 책임·프롬프트·허용 도구의 정의
Loop	`LoopNode` / `LoopIteration`	반복 범위와 특정 회차. Corral의 별명이 아님
Decision	`DecisionNode`	규칙·검토 결과·사람 승인 분기
Run	`CorralRun`	특정 정의 버전과 입력으로 시작한 실행 한 번
Agent execution	`ExecutionInstance`	특정 역할의 실제 실행 시도. 노드와 1:N
Task	`Task`	목표·담당·완료 기준을 가진 업무. 실행 시도와 구분
Reins	`ReinsPanel`	선택한 실행/노드/Run의 제어 패널 이름
Steer	`SteeringDirective`	새로운 지시를 앞으로의 모델 요청에 전달하는 조작
Run settings change	`RunPatch`	해당 Run의 미래 실행에만 적용되는 설정 변경
Approval	`ApprovalRequest`	특정 작업·버전·변경 제안에 대한 사람의 결정
Task Board	`TaskProjection`	공유 작업판. `Hitching Post`로 API를 부르지 않음
Artifacts	`ArtifactVersion`	제출물·초안·검토 결과의 불변 버전
Timeline	`RunEvent`	실행·메시지·개입의 시간순 기록

제품/UI 용어	코드의 정식 타입·단위	의미와 경계
Collaboration	`CollaborationMessage`	질문·답변·제출의 업무상 교환

`Wrangle`는 소개 문구의 동사로만 사용한다. UI에는 `Steer`, `Pause`, `Resume`, `Cancel`, `Change settings`, `Approve`, `Reject`처럼 행동이 분명한 라벨을 쓴다. `Reins mode`처럼 별도의 숨은 조작 모드를 만들지 않는다. `Trough`·`Hitching Post`는 문서/DB/API의 정식 이름으로 채택하지 않는다.

패키지와 CLI: 모노레포 내부에서 `@corral/core`, `@corral/react` 등을 사용한다. npm scope, `corral` 실행 파일명, 도메인·상표의 독점 사용 가능 여부는 확인하지 않았다. 내부 패키지는 `private: true`로 시작하며 공개 발행 직전에 충돌을 확인한다. 이 문서는 이름을 확보했다는 의미가 아니다.

3. 범위

3.1 MVP 필수

영역	필수 제공
편집	실제 노드 D&D·연결·삭제·속성 편집·저장·불러오기·복제·Undo/Redo
그래프	Agent / Decision / Loop / Output. 검토 Loop 1개, 중첩 없음. 실행 입력은 Corral 설정
협업	고정 담당 작업, 질문·답변·결과 제출, 명시된 상대 역할, 동적 자식 호출
산출물	Markdown/일반 텍스트와 구조화된 JSON 검토 결과. 불변 버전·diff·읽은 버전 기록
관찰	설계 그래프의 상태 겹침, 실제 실행 트리, 협업 로그, 대기 대상, 사용량
Reins	실행 중 지시 접수, 전체 pause/resume, 전체·개별 cancel, 미래 설정 변경, 승인·반려
제한	공유 호출·토큰·소환·깊이·질문·시간 제한, 신규 효과 실행 전 검사
연결	Pi agent-core, 서로 다른 공급자 모델 프로필 2개. 사용자 API 키 직접 사용
저장	정의·자료·실행 로그·산출물·변경 기록. 실행 종료 후 외부 호출 없는 재열람
품질	mock 통합 시나리오, 실모델 계약 스모크, 접근성·부하·취소 경합 테스트
공개 데모	API 키 없는 모의 팀. D&D와 Reins에 실제로 반응하는 데모

3.2 이번 MVP에 넣지 않을 것

실행 중 노드 추가/삭제/재연결, 임의 역할 생성, 자유로운 병렬 그래프와 `join:any`, 다중 사용자 편집, 다른 사람의 원격 에이전트, A2A/MCP 확장 로딩, 셸/코딩/파일 동시 수정, 프로세스 재시작 후 자동 재개, 임의 시점의 외부 작업 재실행, 중첩 Corral/Subflow, 자동 판단 라우팅, Local/Jev/Hybrid, LiteLLM, 과거 산출물의 직접 수정은 제외한다.

브라우저 재접속은 지원한다. 이는 **살아 있는 로컬 실행에 다시 연결하는 것**이며, 죽은 실행을 살리는 crash resume이 아니다.

일반 작업 도구는 검색·문서 읽기·산출물 쓰기 3계열이다. 질문·소환·제출 등은 별도의 **협업 프로토콜 도구**다. 3종 도구라는 표현으로 협업에 필요한 인터페이스를 숨기지 않는다.

4. Studio 정보 구조와 UX

4.1 화면

경로(제안)	화면	주요 동작
`/`	Workspace	Corral 목록, 실행 이력, 환경·모델 연결 상태
`/corrals/:id/edit`	Canvas	노드 팔레트, 연결, 속성, 오류 패널, 실행 전 미리보기
`/runs/:runId`	Run	그래프, 실행 트리, Task Board, Timeline, Artifacts, Reins
`/runs/:runId?execution=:id`	선택 실행 상세	입력·모델·도구·대기 이유·적용 지시·산출물
`/runs/:runId?artifact=:id`	버전 상세	본문·diff·원천 입력·검토·관련 개입
`/settings/models`	Model profiles	서버에서 해석되는 두 프로필, 기능·키 존재 여부. 비밀 키 원문 표시 없음
`/demo`	Public demo	모의 실행임을 표시. 로컬 파일·네트워크·실 모델 호출 없음

캔버스는 중앙, 팔레트/실행 트리는 왼쪽, 속성/Reins는 오른쪽, 협업 기록은 하단 또는 탭으로 구성한다. 좁은 화면에서는 탭 전환을 제공하되, MVP의 주 사용 환경은 1280px 이상 데스크톱이다. 1280px 미만에서도 핵심 버튼이 가려져서는 안 된다.

4.2 D&D의 완료 조건

팔레트에서 Agent를 드롭할 수 있고, 포트 연결에 따라 실제 순서가 달라져야 한다. 템플릿의 모델 값만 바꾸는 화면은 D&D 완료로 인정하지 않는다. 빈 캔버스에서 유효한 최소 `Agent→Output`을 만들고 저장·실행할 수 있어야 한다.

연결 드래그 중 유효한 대상 포트와 거절 이유를 표시한다. 연결 후 검증도 같은 계약을 사용한다. nodeId는 복제 시 새로 발급하고, 레이아웃 이동은 정의 hash를 변경하지 않는다. 프롬프트·모델·권한 변경은 정의 revision을 변경한다.

`Undo`는 편집기의 아직 실행되지 않은 정의/레이아웃 변경만 되돌린다. 실행 취소, 외부 API 결과, 이미 제출한 산출물을 되돌리는 기능이 아니다.

4.3 실행 화면

설계 노드가 여러 회차/자식 호출에 대응하므로 “이 노드=실행 하나”라고 표시하지 않는다. 노드에 활성/대기/완료 인스턴스 수를 집계하고 클릭하면 목록을 연다. 실행 트리는 실제 부모·자식 관계를 표시하고, 협업 메시지는 별도 연결 관계로 보여준다.

완료율은 근거 없이 73% 등으로 계산하지 않는다. 단계명, Loop 회차, 완료된 필수 작업 수, 현재 활성 모델 요청 수를 보여준다. 텍스트 스트리밍과 제어 상태 배지는 서로 독립적으로 갱신한다.

4.4 Reins UX

제어 패널 상단에 **대상 / 적용 시점 / 영향 범위**를 항상 표시한다. 선택한 실행의 취소를 Run 전체 취소와 같은 위치·문구로 혼동시키지 않는다. 전체 취소는 상단의 고정 동작, 개별 취소는 선택 항목의 조작이다.

예시: `Target: Writer · execution w-02` → `Delivery: next model request` → `Scope: this execution only`.

설정 변경은 `Current → Proposed` diff와 이미 소비한 예산·미래 대상 수를 보여준 뒤 확인한다. 적용 불가/기한 만료/취소로 미전달 상태도 성공 상태와 동일한 수준으로 보이게 한다.

5. 그래프의 실행 의미

5.1 네 노드

노드	설정	실행 결과
Agent	roleId, invocation, 입력 바인딩, 출력 계약, 모델 프로필, 도구/협업 허용 목록	Task와 ExecutionInstance를 만들고 제출물을 다음 단계에 전달
Decision	`review` / `approval` / 제한된 `rule` 중 하나	이름이 있는 분기 결과. 임의 JavaScript 평가 없음
Loop	본문 노드, 진입/반복/완료 포트, 자동 회차 상한	특정 회차 입력으로 본문 실행. 반복 판단은 구조화 결과만 사용
Output	제출할 버전의 바인딩, 최종 승인 여부	필수 작업·판정·승인 검증 후 최종 버전 확정

Agent의 `invocation`은 `scheduled` 또는 `callable`이다. callable은 실제 그래프 순서에서 자동 실행하지 않으며, 다른 Agent가 허용된 관계로 호출할 때만 실행한다. 팔레트상 새 노드 종류가 아니라 Agent의 속성이다.

5.2 연결 종류

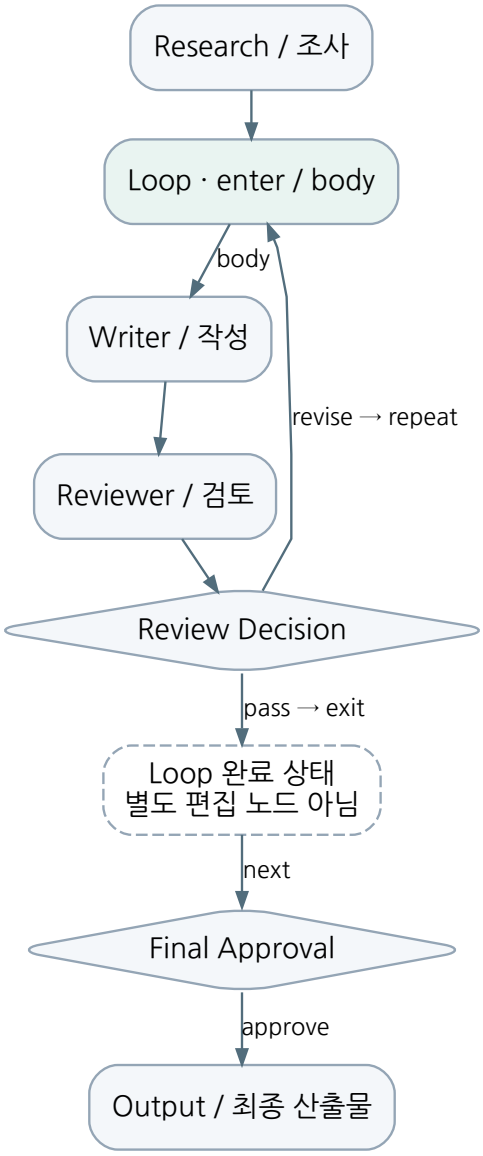
Control edge: 다음 단계 실행을 뜻한다. Agent/Decision/Loop/Output의 정해진 포트 사이에 연결한다.

Capability link: `delegate` 또는 `question`을 허용하는 관계다. 실행 순서를 뜻하지 않는다. 별도 선 모양·핸들·레이어로 표시한다. 실제 호출은 Timeline과 실행 트리에 별도로 나타난다. 허용된 선이 있다는 이유만으로 호출됐다고 표시하지 않는다.

이벤트 스트림과 데이터 산출물의 연결을 control edge에 혼합하지 않는다. 입력 바인딩은 해당 노드 속성에서 선택한다. 범용 타입 설계기는 없지만, `text`, `notes`, `draft`, `review` 등 지원 계약의 호환성 검사는 한다.

5.3 Loop 포트 규약

Loop에는 외부 진입 `enter`, 본문 시작 `body`, 내부 재시도 `repeat`, 내부 통과 `exit`, 외부 다음 단계 `next` 포트를 둔다. 본문에 포함된 노드는 `loopId`를 가진다. 중첩 Loop는 금지한다.



흐름 관계를 문서용으로 재배치한 도식 · 편집용 Mermaid 원본은 ZIP의 diagrams/에 포함

위 그림의 `Loop completed`는 설명을 위한 상태이며 별도 편집 노드가 아니다. 실제 정의에서는 Decision의 pass가 같은 Loop의 exit 포트로 들어간다. compiler는 이 순환을 **회차 1회 실행 함수 + 상한 검사**로 변환한다. 임의 DFS로 화살표를 따라 무한 순환시키지 않는다.

5.4 실행 전 검증

스키마/버전·중복 ID·존재하지 않는 참조·지원하지 않는 노드·포트 방향·출력 포트의 중복 control 연결·진입점/종료점·도달 가능성·Loop 외 순환·본문 경계 침범·누락 상한·허용 목록·입력 생산자·review 계약·모델 기능 호환을 검사한다. callable 노드는 control edge가 없어도 유효한 capability 경로가 있으면 고립으로 판정하지 않는다.

runtime에서도 같은 한도·권한을 재검사한다. 편집기 검증은 신뢰 경계가 아니다. 버전이나 필드를 이해하지 못하는 import는 조용히 무시하지 않고 오류로 반환한다.

6. 대표 협업 시나리오

1. 사용자가 Research, Writer, Reviewer, Review Decision, Loop, Final Approval, Output을 연결한다. Research가 Research Helper를 호출할 수 있도록 capability link를 만든다.

2. Run 시작 시 정의·프롬프트·허용 도구·모델 해석·입력 자료의 스냅샷을 고정한다.
3. Research가 Helper 두 개를 병렬 소환한다. 부모는 자식 대기 상태지만 모델 실행 슬롯을 점유하지 않는다.
4. 두 결과를 읽고 Research가 `notes@1`을 제출한다. 단순 초안 쓰기가 아니라 제출이 완료돼야 Writer에 전달한다.
5. 회차 1의 Writer가 특정 근거에 관해 Research 역할에 질문한다. 이미 끝난 Research 실행을 몰래 다시 열지 않고, 질문에 답할 새 작업·응답 실행을 생성한다.
6. 응답자가 답변과 필요한 근거 버전을 제출한다. Writer가 `draft@1`을 제출하고 Reviewer가 그 정확한 버전을 검토한다.
7. `review@1`이 revise이면 수정 작업을 만들고 다음 회차에 `draft@1`과 `review@1`을 전달한다.
8. 사용자가 Writer 노드의 미래 프롬프트에 “1차 자료만 근거로 사용”을 추가한다. 완료 기준도 바꾸는 경우 별도의 criteria 패치를 확정한다. 그 이후의 요청 스냅샷에 포함된 ID가 기록된다.
9. Writer는 `draft@2`, Reviewer는 **이번에 작성된** `draft@2`에 대한 `review@2`를 만든다. 이전 review 입력과 현재 review 출력을 섞지 않는다.
10. 통과한 특정 버전을 사용자가 최종 승인하면 Output이 확정된다. 취소·한도·실패 경로는 같은 정의의 별도 실행 테스트로 검증한다.

이 시나리오에서 모델이 반드시 첫 회차에 반려하고 두 번째에 통과할 것이라고 가정하지 않는다. 고정 데모/테스트는 MockAdapter로, 실제 LLM은 별도의 품질 평가로 검증한다.

7. 협업과 산출물 계약

7.1 Task와 실행 시도

Task 담당 역할은 한 명이다. 동시에 활성화된 ownerExecutionId도 최대 하나다. 동일한 Task를 재시작하더라도 이전 실행을 덮어쓰지 않고 attemptNo를 증가시킨 새 ExecutionInstance를 연결한다. 자동 재할당·경매·작업 훔치기는 제공하지 않는다.

작업 상태는 `queued / in_progress / blocked / submitted / completed / failed / cancelled`다. submitted는 산출물을 냈다는 사실, completed는 해당 Task의 완료 계약을 충족했다는 사실이다. 그래프 전체의 검토 통과나 Run 성공과 같은 의미가 아니다.

7.2 협업 메시지

타입	필수 필드	선택 필드	종료 규칙
question	taskId, senderExecutionId, targetRoleId, questionId, body, deadline	artifactRefs	답변/취소/타임아웃으로 반드시 종결
answer	questionId, responderExecutionId, taskId, body	evidenceRefs	열린 질문 하나에 최종 답변 하나. 중복은 멱등 처리
submission	taskId, executionId, artifactVersionIds	comment	자신이 소유한 작업의 허용 산출물만 제출

사람의 지시는 동일한 실행 감사 체계에 남지만 agent answer로 위장하지 않는다. `actor.kind=user`의 별도 command/directive다.

`question`은 허용 관계를 검사한 뒤 새로운 응답 Task를 만든다. 그 Task는 질문을 보낸 실행의 소유 하위 작업이므로 취소 전파 경계가 명확하다. 대상 역할의 다른 활성 실행에 임의로 대화를 끼워 넣지 않는다. 질문 생성도 spawnCount와 depth에 포함한다. depth는 control graph 깊이가 아니라 **동적으로 생성한 실행 계보**를 센다.

7.3 대기와 교착

모델 호출 permit은 모델 통신 동안만 점유한다. `ask`, `await_tasks`로 기다리는 부모는 permit을 반환한다. resident worker 수의 한도와 모델 동시성의 한도를 분리한다.

대기 관계는 `{kind, targetIds, since, deadline, reasonCode}`로 저장한다. 신규 wait edge가 순환을 만들면 실패 결과를 반환한다. 새 인스턴스를 계속 만드는 질문 사슬은 단순 순환 탐지로 잡히지 않을 수 있으므로 총 질문/소환/깊이/시간 제한도 병행한다.

7.4 산출물

`artifactId + version`은 불변이다. 같은 이름이어도 두 병렬 작업의 결과는 taskId 또는 artifactId로 구분한다. 전역 `notes-part@latest`에 두 helper가 덮어쓰지 않는다.

write는 새 버전을 저장하고 submit은 작업의 공식 출력을 확정한다. 취소된 시도의 미제출 버전은 `unsubmitted/abandoned`로 남길 수 있으나 다음 control 단계에 자동 전달하지 않는다. 생성된 산출물을 취소 때문에 삭제하거나 이전 버전으로 둔갑시키지 않는다.

노드 시작 시 입력 참조를 해석하고 고정한다. 바인딩은 `run_input`, `task_output`, `previous_iteration_output`, `question_answer`, `explicit_version`만 지원한다. 데이터 전달의 세부 형식은 기술 설계 및 예제 JSON을 따른다.

7.5 검토

검토 결과에는 `targetArtifactVersionId`, `criteriaVersionId`, `verdict: pass|revise|inconclusive`, `issues[]`, `evidenceRefs[]`를 요구한다. revise에는 최소 하나의 수정 사유가 필요하다. 형식 오류·대상 버전 불일치·기준 버전 불일치는 pass로 처리하지 않는다.

일반 Steer는 완료 기준을 자동 변경하지 않는다. 사용자가 별도의 기준 변경을 확정하면 CriteriaRevision을 새로 만든다. 일반 작성 방식 지시와 완료 기준 수정은 UI에서 구분한다. 기존 검토는 삭제하지 않고 `review_stale` 표시 후 다음 허용 회차의 재검토를 요구한다. 기준 변경은 Loop 종료 전까지만 가능하며 Final Approval 이후에는 새 Run이 필요하다.

8. Reins — 실시간 개입 명세

8.1 조작과 적용 경계

명령	기본 대상	적용 경계	하지 않는 일
`steer.enqueue`	선택 실행 1개	다음 모델 요청의 컨텍스트 확정 시점	진행 중 응답의 내용을 순간 수정
`run.pause` / `run.resume`	Run 전체	신규 효과 접수를 막고 진행 중 요청·도구 정리 후 안전 경계	OS/프로세스 중단 후 복구
`execution.cancel`	선택 실행 + 소유 후손	신규 효과 차단 후 Abort·종료 확인	자동 재할당·이미 수행한 외부 작업 롤백
`run.cancel`	전체 Run	모든 활성 범위의 취소 정리	즉시 cancelled로 표시

명령	기본 대상	적용 경계	하지 않는 일
`execution.restart`	취소된 미완료 Task의 실행 시도	기존 시도 종료 확인 후 새 실행	완료한 노드를 과거 시점에서 재실행
`run.patch`	nodeId의 미래 활성화 또는 Run 상한	명시한 경계 이후	실행 중 그래프 구조·도구 권한 확대
`approval.resolve`	특정 approvalId	승인 대상 revision 일치 시 한 번	예산을 늘리지 않고 승인만으로 우회

재시작은 부모가 아직 해당 필수 작업을 기다리고 있고, 그래프가 이미 그 작업 이후로 진행하지 않은 경우만 허용한다. 조건을 만족하지 않으면 새 Run 또는 다음 검토 회차를 사용한다. 재시도는 같은 taskId, 새 executionId이며 이전 결과와 지시를 명시적으로 연결한다.

8.2 지시 전달 상태

`accepted → queued → attached_to_request → delivered`를 기본으로 한다. `delivered`는 해당 입력을 포함한 요청을 로컬 transport가 시작했다는 뜻이며 모델이 이해/준수했다는 뜻이 아니다. 요청 전 취소는 `not_delivered`, 대상 종료는 `expired`, 검증 실패는 `rejected`다. 응답 전 네트워크 단절은 `delivery_unknown`으로 남길 수 있다.

두 지시가 같은 요청에 포함되면 순서와 두 ID를 기록한다. MVP에서는 이미 접수한 지시를 과거 기록에서 삭제하지 않는다. 방향을 정정하려면 새로운 명시적 상쇄 지시를 보낸다. 대기 지시 자체의 철회 UI는 후속으로 두되, 취소/종결로 인한 미전달 상태는 표시한다.

8.3 설정 패치

설정 변경 대상은 `roleId` 전체가 아니라 **선택한 nodeId의 미래 Execution 활성화**가 기본이다. 같은 Role을 다른 노드에서 재사용해도 그 노드는 바뀌지 않는다. callable 노드 변경은 이후 그 노드를 호출하는 응답/자식 실행부터 적용한다.

Loop 회차의 최초 상한은 `LoopNode.maxIterations` 한 곳에 둔다. `loop_limit` 패치가 해당 nodeId의 다음 회차 진입부터 이를 덮는 run-only 값을 만든다. 전역 limits에 같은 maxIterations를 중복 저장하지 않는다.

한 Run에는 baseDefinitionHash와 단조 증가 controlRevision이 있다. 패치는 이전 revision과 대상·시점·변경 키를 포함한다. 허용 키는 모델 프로필, 역할 프롬프트, 루프 자동 회차, 소환·호출·토큰 예산뿐이다. 권한·연결·노드·도구 구현 변경은 거절한다.

실행 전체의 토큰/소환 한도는 즉시 미래 admission부터 적용한다. 사용량은 초기화하지 않는다. 설치 정책의 hard cap을 넘기는 변경은 현재 Run에서 허용하지 않는다.

8.4 안전한 일시정지

pause 접수 시 새 모델 호출·새 도구·새 Task 소환의 admission을 닫는다. 이미 시작한 모델/도구는 유한 timeout 안에 마무리하거나 사용자 요청에 따라 취소한다. 질문/자식 대기 중인 worker는 안전 경계로 간주할 수 있으며 답변이 올 때까지 전체 정지가 지연되어서는 안 된다.

Run이 실제 `paused`가 되는 조건은 **활성 외부 효과가 0이고, 모든 worker가 경계 또는 종결 상태**인 것이다. `pausing` 동안 지시 접수·취소·로그 열람·예산 변경은 허용한다. 정상 실행 작업은 정지한다. pause와 resume은 멱등 명령이다. resume은 자동 승인을 뜻하지 않는다.

8.5 취소의 완료와 한계

취소 의도 접수 직후 대상 scope의 새 admission을 차단한다. 이미 admit된 요청은 취소가 전달될 때까지 잠시 살아 있을 수 있다. 원격 공급자 내부의 처리·청구까지 중단되었음을 보장하지 않는다.

모든 소유 후손 worker, backend tool 요청, 대기 promise가 종료 또는 취소 결과로 정리돼야 `cancelled`를 확정한다. timeout 후 worker를 강제 종료하면 `forced` 여부를 기록한다. 종료 확인을 못한 경우 완료라고 숨기지 않고 `termination_unconfirmed` 실패로 표시한다. 늦게 도착한 사용량은 정산 기록으로만 추가하고 Task/Run을 부활시키지 않는다.

부분 취소로 필수 Task가 사라지면 `awaiting_user`로 이동한다. 선택지는 **새 시도로 다시 시작** 또는 **Run 취소**다. "없어도 성공한 것으로 진행"은 MVP에 없다. 질문 응답 작업을 취소한 경우에는 질문자에게 구조화된 취소 결과를 전달한다.

8.6 인과 기록

산출물에는 생산 modelCallId, contextSnapshotId, 실제 전달된 directiveIds, 적용 configRevision, 읽은 입력 버전 ID를 연결한다. 이것은 **프로그램이 관측한 데이터 경로**이며 특정 문장이 어느 지시 때문에 생겼는지에 대한 심리적 인과를 증명하는 것은 아니다.

사용자 지시가 10:01에 접수됐어도 10:00에 시작한 요청의 10:02 결과에 그 지시를 직접 원인으로 붙이지 않는다. 다음 요청에 실제로 포함된 시점부터 직접 적용 연결을 만든다. 간접 전달은 산출물 참조 경로로 별도 표시한다.

9. 제한·예산의 초기값

아래 값은 **이번 계획에서 정한 출발 설정**이며 성능 측정이나 비용 최적화 결과가 아니다. 상한을 올릴 수 있더라도 기록·누적 사용량·hard cap은 유지한다.

항목	기본값	hard cap / 조절
캔버스 노드 수	최대 20	MVP 고정
Loop 수·중첩	1 / 중첩 없음	MVP 고정
Loop 자동 회차	2	사용자 패치로 10까지
동적 위임 깊이	2	MVP 고정. 최초 scheduled agent=0
동시 모델 요청	3	MVP 고정. 대기는 슬롯 사용 안 함
실행당 동적 생성	12	사용자 패치로 32까지. 질문 응답 생성 포함
실행당 모델 요청	40	사용자 패치로 100까지. 실패·재시도 포함
총 출력 토큰 예산	64,000	사용자 패치로 256,000까지. 알려진 지원 모델만 강제
요청별 출력 토큰 상한	4,096	모델 기능과 잔여 예약 예산 안에서 결정
실행당 질문	6	고정. 한 실행의 열린 질문 최대 1개
질문 대기	120초	user pause 동안 논리 대기 타이머를 멈추되 전체 wall deadline은 계속
모델/문서 도구 timeout	180초 / 30초	취소 정리 timeout 별도 10초

항목	기본값	hard cap / 조절
전체 wall deadline	60분	사람 대기 포함. 만료하면 신규 실행 차단·정리
산출물/입력 문서	버전당 UTF-8 1 MiB	workspace 총량 제한 별도
무진전	6턴	단순 메시지 왕복은 진전으로 간주하지 않음

예산은 요청 전 예약하고 종료 후 실제 usage로 정산한다. 취소 등으로 실제 usage가 없으면 예약량을 임의로 0으로 환급하지 않고 미확인 항목으로 남긴다. 모델이 엄격한 토큰 상한을 지원하지 않으면 UI가 이를 hard cap으로 주장해서는 안 된다.

금액은 확인된 모델 단가 스냅샷이 있을 때만 추정하며, provider 청구서와 같은 값이라고 부르지 않는다. 캐시 읽기/쓰기, 입력/출력/추론 토큰 등 공급자가 반환한 범주를 중복 집계하지 않도록 provider별 usage normalization 테스트를 둔다.

10. 기록·보안·비기능 요구

10.1 저장과 재열람

SQLite에 정의 revision, 레이아웃, 실행 이벤트, Task/Execution 투영, 입력 스냅샷, 산출물, 개입과 승인, usage를 저장한다. 텍스트 산출물도 같은 DB 트랜잭션에서 저장하여 로그만 남고 본문이 사라지는 이중 쓰기를 피한다.

프로세스 재시작 시 남아 있는 running/paused Run은 `interrupted`로 확정한다. 자동 provider 호출·outbox 재전송은 하지 않는다. 기록을 읽는 동작에는 모델·검색 키가 필요하지 않다. 입력·프롬프트 스냅샷을 포함하지 않은 내보내기는 완전한 재구성 파일이라고 부르지 않는다.

10.2 기본 보안

localhost만 bind하더라도 임의 웹페이지의 요청을 신뢰하지 않는다. 시작 시 발급한 세션, Host/Origin 검사, 상태 변경 요청의 CSRF 방어, rate/size 제한을 적용한다. API 키는 backend 환경/키 저장소에서만 읽으며 Corral JSON·클라이언트 번들·이벤트 payload에 저장하지 않는다.

MVP worker에는 read/write shell 도구가 없고, 승인된 원격 모델 요청 외 기능은 Coordinator RPC를 거친다. 프로세스 분리는 정리·장애 격리를 위한 것이며 완전한 보안 sandbox라고 주장하지 않는다.

문서 읽기는 workspace의 등록된 documentId 또는 검증된 HTTPS sourceId만 받는다. 임의 로컬 경로를 받지 않는다. 외부 URL은 SSRF 검증과 크기/타입/redirect 제한을 거친다. Markdown 출력은 HTML 실행 없이 렌더링하고 위험한 URL을 막는다. 원격 이미지 자동 로드는 기본 비활성화한다. [S24]

10.3 접근성·성능 목표

표시 상태는 색만으로 구분하지 않는다. 이름·아이콘·텍스트를 함께 사용한다. 키보드로 노드 추가/선택/이동/연결/삭제 및 Reins를 사용할 수 있고, 긴 한국어·일본어 지시 입력 중 IME Enter가 전송을 일으키지 않는다.

측정 목표는 출시 시 검증할 수치이지 이미 달성한 성능이 아니다. production build, 고정된 브라우저·화면·테스트 장비를 기록한다. 20개 설계 노드, 100개 과거 실행 인스턴스, 10,000개 이벤트, 초당 50개의 수신 프레임 부하에서 로컬 제어 접수 p95 200ms, 영속 이벤트 반영 p95 300ms를 목표로 한다. 실제 모델 응답 시간은 제외한다. 캔버스 드래그 중 이벤트 증가로 지속적인 전체 재렌더가 생기면 실패다.

11. 수용 기준과 출시 경로

AC	검증할 장면	연결 요구
AC-01	빈 캔버스에서 노드를 드롭·연결하고 설정 폼만이 아닌 실제 흐름 변경을 실행	CAN-001, CAN-004
AC-02	저장/불러오기/복제 및 레이아웃 이동 시 실행 정의 hash 불변	CAN-002, CAN-005
AC-03	조사자의 두 자식과 depth 2 손자가 실행 트리에 표시	RUN-002, RUN-004
AC-04	Writer 질문→응답 작업→답변이 작업·버전 참조로 연결	COL-002, COL-003
AC-05	draft@1 반려→다음 회차 draft@2→draft@2 검토를 정확히 연결	ART-002, ART-003
AC-06	화면만으로 대기 이유·원천 입력·생성 경로·usage를 확인	OBS-001, OBS-004
AC-07	Loop/소환/모델 호출/토큰 상한이 조용히 넘어가지 않음	RUN-006
AC-08	진행 중 지시는 접수되지만 이전 요청의 결과에는 적용됐다고 표시하지 않음	RNS-001, RNS-002, RNS-010
AC-09	미래 노드 모델·프롬프트 변경이 기존 실행/템플릿을 덮지 않음	RNS-006, RNS-009
AC-10	상한 증가·감소 시 누적값·예약량 유지 및 신규 admission 정책 반영	RNS-007
AC-11	pause 중 신규 실행 없음, resume 이후 정상 계속, 자동 승인 없음	RNS-003
AC-12	전체 취소가 후손과 backend 도구를 정리하며 종료 전 완료 표시 없음	RNS-004, SEC-004
AC-13	개별 취소로 질문자/부모가 무한 대기에 남지 않음	COL-004, RNS-004
AC-14	조건부 재시작이 새 실행 ID를 사용하고 기존 완료 노드를 재실행하지 않음	RNS-005
AC-15	중복 명령·승인·늦은 결과·설정 revision 충돌을 올바르게 처리	DEV-001, RNS-008
AC-16	SSE 재접속·탭 재열기 후 같은 투영. backend 재시작 후 자동 외부 호출 없음	OBS-002, RUN-007
AC-17	악성 import/권한 없는 도구/로컬 접근 URL/렌더링 스크립트 차단	SEC-001~003
AC-18	실제 모델 프로필 두 개의 요청·usage·취소 계약 스모크	DEV-004, OBS-004

AC	검증할 장면	연결 요구
AC-19	키 없는 공개 데모와 키보드/IME·부하 시험	DEV-005, UX-001, UX-002
AC-20	요구→테스트→태스크 추적과 모든 필수 품질 게이트 통과	DEV-003, DX-002

정상 완료·전체 취소·개별 취소·예산 소진은 각각 독립된 실행으로 검증한다. 하나의 Run이 성공하면서 동시에 중간 취소까지 완성해야 한다는 모순된 수용 기준을 만들지 않는다.

12. CLI와 개발자 경험

아래는 구현할 CLI 계약이다. **현재 이미 설치 가능한 명령이라는 뜻이 아니다.**

```
corral dev                          # 로컬 Studio와 제어 서버
corral dev --demo                   # 모의 실행 프로필
corral validate workflow.corral.json
corral run workflow.corral.json --input brief.md --watch
corral inspect <run-id>             # 저장 기록 열람, 외부 호출 없음
corral export <run-id> --redact     # 공개 가능한 기록 묶음
```

CLI 실행도 같은 Coordinator를 사용한다. 승인 대기 시 --watch 터미널과 Studio 중 어느 쪽에서든 동일한 approvalId를 해결할 수 있다. headless에서 사람이 없는데 승인을 자동 허용하지 않는다. SIGINT는 전체 취소 요청이며, CLI가 사라졌다고 서버에 남은 Run을 모르고 방치하지 않도록 명시적 detach 옵션을 별도 설계한다.

13. 이후 확장

v0.2는 **Local/Jev의 수동 교체 가능한 DecisionProvider**를 구현한다. 질문 종류·후보·응답 검증이 같은 범위에서만 전환하고, 성능 우열이나 자동 절감은 별도 평가한다. v0.3은 Corral 중첩/재사용과 더 넓은 그래프, 이후는 외부 AgentAdapter/A2A·LiteLLM·crash resume을 다룬다.

현재 스펙의 MVP 게이트를 후속 기능으로 대신하지 않는다. 비용 최적화가 없더라도 D&D와 협업·Reins를 검증하면 MVP는 성립한다.

구현상 세부 계약은 Technical-Design.md, 요구사항 전체는 requirements.json, 테스트 설계는 Test-Strategy.md에 있다.

PART 02

기술 설계

상태 소유 · Coordinator · GraphRunner · Pi · 저장 · 이벤트

> 이 문서는 제품 스펙을 구현 가능한 책임·데이터·전이·통신 계약으로 분해한 설계안이다. 라이브러리 조합의 동작을 검증했다는 보고서가 아니다. 아래 인터페이스는 Corral이 구현할 포트이며, 실제 Pi/LangGraph API를 그대로 인용한 코드가 아니다.

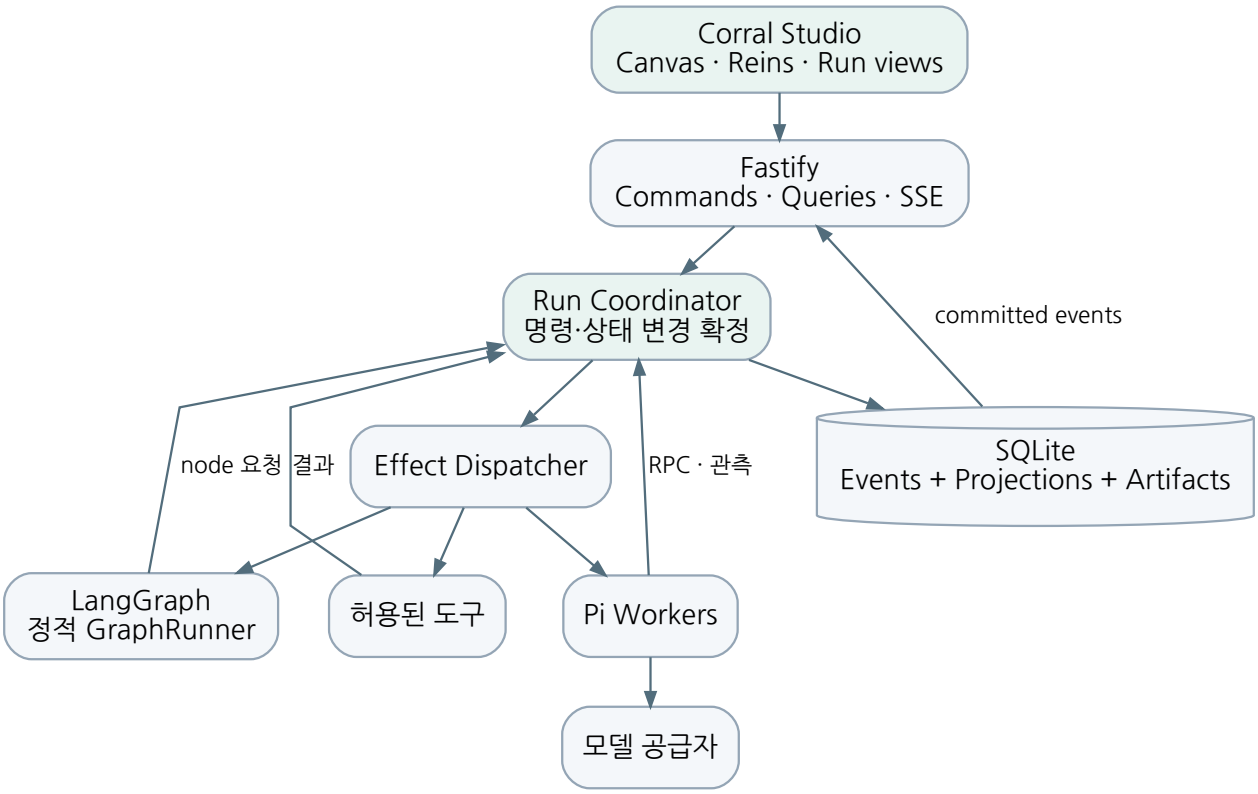
1. 아키텍처 결정

React/Vite/React Flow + Fastify + Run Coordinator + LangGraph.js 어댑터 + Pi agent-core worker + SQLite를 채택한다. LangGraph와 Pi는 제품 상태의 독립 소유자가 아니다. XState·Redux·Temporal·Redis·LiteLLM을 동시에 추가하지 않는다.

LangGraph는 검증된 정적 control flow를 순회하고 노드 실행을 Coordinator에 요청한다. Task·메시지·개입·예산·산출물·Run terminal 상태는 Coordinator가 확정한다. LangGraph의 실행 반환은 성공의 제안이며, 필수 Task·승인·활성 후손 검증 없이 Run을 성공시킬 수 없다.

MVP에서는 LangGraph의 영속 checkpointer와 `interrupt()`에 제품의 스티어링을 맡기지 않는다. 제품의 pause/approval은 Coordinator의 취소 가능한 경계 대기다. 그래프 state에는 runId, 참조 ID, 제어 결과만 두고 대화·메시지·Task 보드를 복제하지 않는다. 재시작 후 실행 복구가 없으므로 이 경계가 가능하다. LangGraph의 interrupt는 재진입 시 노드 앞부분을 다시 실행할 수 있어, 장수 Pi 세션의 중간 정지를 동일시하지 않는다. [S05]

1.1 구성도



흐름 관계를 문서용으로 재배치한 도식 · 편집용 Mermaid 원본은 ZIP의 diagrams/에 포함

1.2 상태 소유 표

데이터	쓰기 권한	캐시/사본	금지
정의·레이아웃	DefinitionService 트랜잭션	편집기 draft	live Run에 draft를 직접 덮기
Run/Task/Execution/ Approval	Coordinator	SQL projection, UI projection	Pi/GraphRunner가 직접 terminal 확정
event seq·controlRevision	Coordinator	UI watermark	timestamp로 seq 대체
외부 요청 permit·예산 예약	Coordinator	worker의 단회성 permit	worker가 자식 예산 새로 만들기
대화 컨텍스트	요청 전 Coordinator가 스냅샷 확정, Pi는 실행용 사본	Pi 세션	과거 요청 snapshot 수정
스트리밍 텍스트	worker 관측 → 스트림 채널	UI buffer	토큰 delta로 도메인 완료 추론
산출물 본문·버전	ArtifactService, Coordinator tx 내부	읽기 전용 UI	동일 version UPDATE

이벤트 로그가 영속 이력의 기준이며, 현재 상태용 SQL projection은 같은 트랜잭션에서 갱신한다. 별도 event store 서비스나 분산 CQRS 플랫폼은 도입하지 않는다. projection을 replay해 동일 상태가 되는지 검사한다.

2. 모노레포와 의존 방향

```
corral/
  apps/
    studio/                 # React/Vite. Node/Pi/DB import 금지
    server/                 # Fastify, 세션, REST/SSE
  packages/
    contracts/              # Zod schema, types, errors, generated JSON Schema
    core/                   # Coordinator, limits, tasks, messages, pure reducers
    runner-langgraph/       # 정적 control-flow lowering과 node callback
    adapter-pi/             # parent/worker 프로토콜, Pi API mapping
    storage-sqlite/         # Drizzle schema, migrations, repositories
    react/                  # Canvas/Reins/RunView의 내부 재사용 컴포넌트
    testkit/                # clock/id/agent/search/store test doubles
    cli/                    # corral dev/run/validate/inspect/export
  specs/                    # feature spec, acceptance.feature, traceability
  tests/{unit,contract,integration,e2e,live,performance}/
  examples/{research-team,fixtures}/
  docs/{adr,dependencies,runbooks}/
```

`contracts ← core ← adapter/runner/storage/server`의 방향을 지킨다. contracts/core는 DOM·React에 의존하지 않는다. `@corral/react`는 contracts와 frontend용 순수 selector만 참조한다. DB 구현이 runtime 밖으로 흘러나오지 않도록 Repository 포트를 사용한다.

패키지는 초기에 내부 workspace만 사용한다. 공개 SDK 버전 호환성을 약속하거나 매 패키지를 npm에 배포하지 않는다. package names는 발행 가능 여부 미확인이다.

3. 식별자·revision·시간

필드	의미
`corralId`	저장한 정의의 안정 ID
`definitionRevision`	의미 있는 정의 편집 순번
`definitionHash`	정규화한 정의·내장 프롬프트의 SHA-256
`layoutRevision`	좌표·크기 등 화면 편집 순번, definitionHash와 독립
`runId`	실행 한 번
`controlRevision`	사용자가 패치/개입 정책을 변경할 때의 revision. token delta마다 바뀌지 않음
`runSeq`	모든 영속 RunEvent의 순서, Run 내부에서 단조 증가
`nodeId` / `roleId`	배치한 노드 / 역할 정의
`taskId`	완료 대상 업무
`executionId` / `attemptNo`	실제 시도 / 동일 업무의 시도 번호
`parentExecutionId`	취소·소유 계보, 루프 그룹 깊이와 무관

필드	의미
`iterationId` / `iterationNo`	검토 반복의 실행/회차
`modelCallId` / `toolCallId`	요청/도구 실행. retry는 새 ID
`commandId`	클라이언트가 생성하는 멱등 명령 ID
`effectId`	외부 효과의 단회 실행 ID
`contextSnapshotId`	그 요청이 실제로 사용한 입력·지시·설정
`directiveId` / `patchId`	자연어 지시 / 설정 변경
`criteriaVersionId`	완료 기준의 불변 버전

ID는 주입 가능한 UUID 생성기로 만들고 순서는 ID가 아니라 runSeq로 판단한다. wall 시각은 ISO UTC, timeout은 monotonic clock을 사용한다. UI에서 로컬 시간으로 변환한다. 테스트는 Clock/ID provider를 주입한다.

정규화는 객체 키 정렬·정의용 배열의 안정 순서·UTF-8 직렬화를 명세한다. `NaN`, 함수, undefined, 날짜 객체 같은 JSON 외 타입은 거절한다. 레이아웃·사용자 secret·실행 시각을 definitionHash에 넣지 않는다. 실제 provider/model ID와 단가·기능 해석은 RunSnapshot에 별도로 고정한다.

4. 정의와 compiler

4.1 JSON이 canonical format

기본 확장자는 `.corral.json`이다. D&D 에디터가 이 정의를 생성한다. YAML은 MVP의 필수 입력이 아니다. 코드로 정의를 만들 수 있지만 그것이 유일한 설계 방식이 되어서는 안 된다.

정의에는 `schemaVersion, id, name, entryNodeId, input, roles, nodes, controlEdges, capabilityLinks, limits`를 둔다. layout은 별도 JSON이다. prompt는 MVP에서 정의 안의 문자열로 저장한다. 나중에 파일 참조를 도입할 경우 Run 시작 시 본문까지 snapshot해야 한다.

역할: `id, prompt, modelProfile, allowedWorkTools, outputContract`. Agent node는 `roleId, invocation, inputBindings, outputName, loopId?`를 가진다. 동일 역할의 여러 노드를 구분해 patch를 적용할 수 있다.

4.2 compiler 파이프라인

`JSON parse → version/schema validation → references/ports validation → capability reachability → Loop-boundary validation → input binding validation → IR lowering → plan summary`.

IR은 `AgentStep / DecisionStep / ReviewLoop / OutputStep`와 정적 연결이다. 사용자가 넣은 코드/함수를 eval하지 않는다. rule은 `review_verdict`, `required_tasks_submitted`, `human_approval`의 enum으로만 선택한다. 수식 언어는 MVP 밖이다.

Loop 본문은 `loopId`로 묶는다. 외부 edge는 enter/next, 내부 종료는 repeat/exit만 허용한다. 이 경계를 통과하지 않는 외부↔본문 연결은 거절한다. 각 본문 경로는 Loop로 반환하거나 명시적으로 실패한다. `repeat`가 자동 회차 상한을 넘어가면 새 본문을 호출하기 전에 Approval을 생성한다.

GraphRunner의 node callback은 `requestExecution(runId,nodeId,iterationId)`를 요청한다. 동일 계획 활성화가 실수로 재진입해도 activationKey의 unique 제약으로 Task를 중복 생성하지 않는다. LangGraph node retry는 MVP에서 비활성화하고 재시도는 Corral의 시도·예산 계약으로만 처리한다.

4.3 일반 분기와 병렬의 범위

root에서는 정적 Agent 순서, 승인/검토 분기, Output을 지원한다. 임의 DAG의 모든 fan-out/fan-in 의미는 지원하지 않는다. 병렬성은 명시적 동적 child tasks와 전체 합류(`all`)에서 제공한다. 지원하지 않는 연결은 에디터에서 생성되더라도 검증 오류로 막는다.

이 제한은 "그림은 자유롭게 그리고 실행은 다른 방식으로 한다"는 뜻이 아니다. **지원하는 연결 의미를 UI에 드러내고 그 범위 안의 편집은 모두 실행**한다.

5. Coordinator 명령 처리

5.1 원자적 경계

Run당 직렬 command mailbox를 둔다. 한 명령 처리에서 네트워크/worker completion을 await하지 않는다. 오래 걸리는 작업은 effect로 넘기고, 완료 관측은 새 command로 돌아온다.

```
1. 세션/actor/대상/명령 스키마 확인
2. commandId + payloadHash 조회: 동일 재전송은 기존 receipt 반환
3. expectedControlRevision / target epoch 확인
4. 순수 decide(state, command) → events + effect intents + receipt
5. SQLite transaction:
   command 기록, seq 배정, events INSERT,
   projection 적용, artifact 본문 INSERT, effect intents INSERT
6. commit 성공 후 UI로 영속 events를 publish
7. commit된 effect만 dispatcher에 전달
8. effect result를 새 observation command로 처리
```

commit 실패 시 외부 효과를 시작하지 않는다. storage가 쓰기 불가면 admission을 닫고 활성 효과를 best-effort 취소한다. 영속 기록을 남길 수 없는데 정상 계속하지 않는다.

5.2 event와 command

command는 의도, event는 확정된 사실, effect는 프로그램 밖에서 수행할 일이다. `cancel_requested`는 사실이지만 `cancelled`와 다르다. 모델이 "완료"라고 말한 메시지는 관측이며 Task completion 확정 명령을 대체하지 않는다.

`commandId`가 같고 payload가 다르면 HTTP 409 `IDEMPOTENCY_CONFLICT`. controlRevision이 오래되었으면 409 `STALE_CONTROL_REVISION`. 취소는 같은 scope에 멱등이며 급한 전체 취소를 불필요한 revision 충돌로 막지 않는다. 대상 runId/executionId 존재·권한은 반드시 검사한다.

5.3 외부 효과의 보장

로컬에서 `effect.admitted`를 확정하는 순간이 신규 실행 허용의 선형화 지점이다. 취소 fence 이후 대상 scope의 새 effect를 admit하지 않는다. 취소 이전에 이미 허가된 worker 요청의 실제 network 시작 관측이 나중에 도착할 수 있으므로 시간순 start 이벤트만으로 위반을 오판하지 않는다.

분산 공급자의 '정확히 한 번 실행'은 보장하지 않는다. timeout으로 성공 여부가 불명확한 요청을 조용히 자동 재시도하지 않는다. 읽기 성격의 안전한 요청에만 명시된 재시도 정책을 적용하고 매번 새 modelCallId/예산을 기록한다.

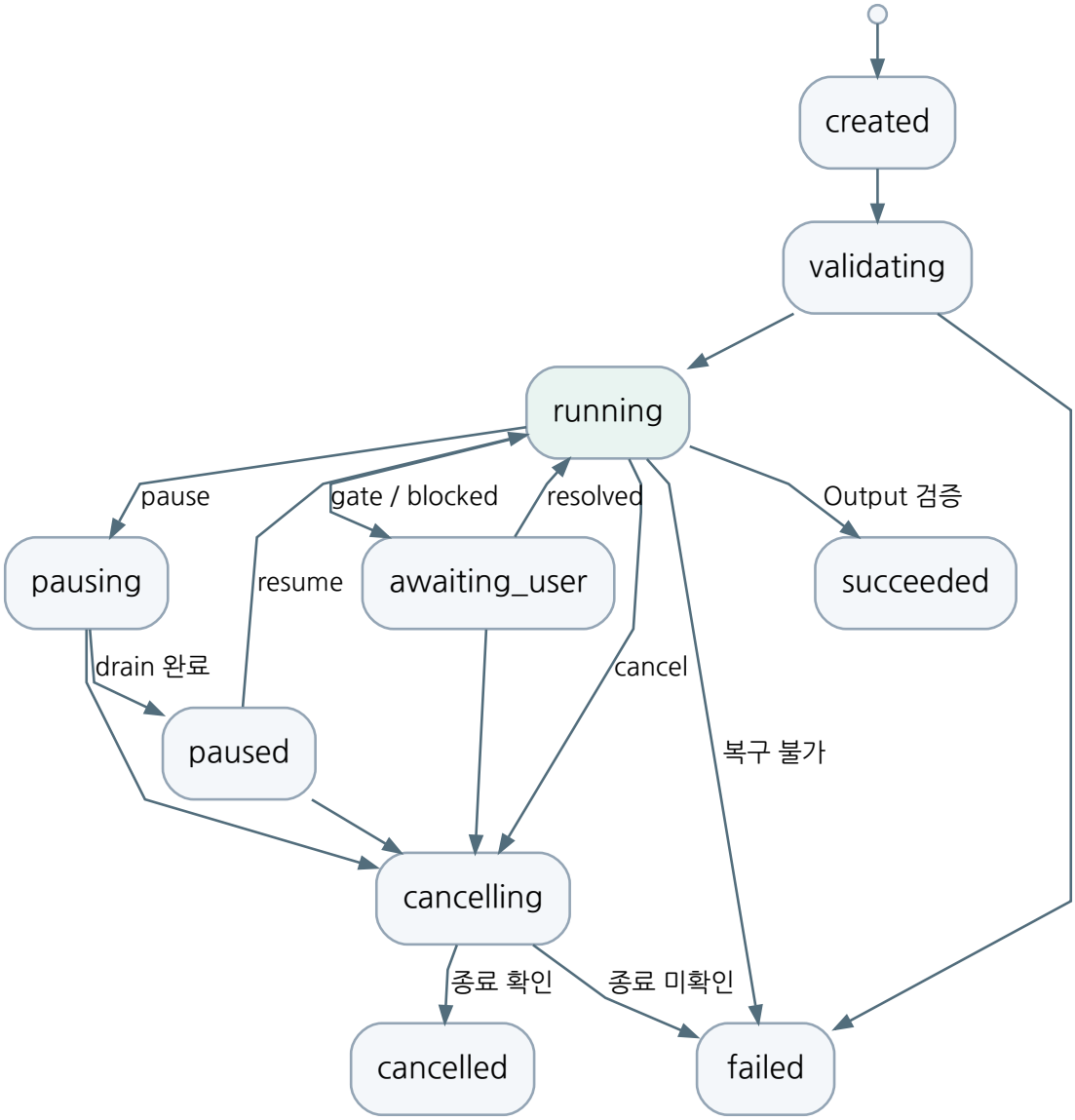
crash 이후 pending effect를 자동 재발행하지 않는다. 실행 기록용 effect intent는 남기지만 recovery 정책은 Run interrupted다. 향후 crash resume을 도입할 때 별도 설계해야 한다.

6. Run·Execution·Task 상태

6.1 Run

`created → validating → running`. 일반 제어 상태는 `running / pausing / paused / awaiting_user / cancelling`. 종료 상태는 `succeeded / failed / cancelled / interrupted`.

Run 상태를 자식들의 단순 OR로 만들지 않는다. 일부 자식이 메시지를 기다리고 다른 자식이 실행 중이면 Run은 running이다. 모든 진행이 승인에 막혔을 때 awaiting_user다. pausing/cancelling은 명시적 사용자 명령의 상태이며 자동 추론하지 않는다.



흐름 관계를 문서용으로 재배치한 도식 · 편집용 Mermaid 원본은 ZIP의 diagrams/에 포함

프로세스 종료 후 비종료 상태에서 interrupted로 전이하는 복구 규칙은 별도이며 diagram의 정상 경로와 구분한다. 이미 종료한 Run은 재시작되지 않는다.

6.2 Execution

`queued / ready / active / waiting_children / waiting_message / waiting_approval / pausing / paused / cancelling / succeeded / failed / cancelled`.

`active`와 함께 `activity=model|tool|finalizing`을 표시한다. 논리 생명주기와 현재 활동을 구분하면 tool-start 이벤트 하나 때문에 전체 Task 상태가 뒤집히는 일을 피할 수 있다. UI 표시 상태는 두 필드를 조합한다.

WaitingOn에는 `kind, targetIds, reasonCode, since, deadline, required`를 둔다. 단순한 자유문장만 저장하지 않아 재접속·로컬라이징·취소 전달에 사용할 수 있게 한다.

6.3 Task

Task는 역할 소유와 실행 시도 소유를 구별한다. `assignedRoleId`는 고정, `activeExecutionId`는 시도마다 바뀔 수 있다. 작업이 blocked인 이유는 부모의 질문·승인일 수 있다. Task를 cancel하면 그 Task의 active attempt와 소유 후손을 cancel한다.

명시적 restart는 cancelled/failed인 시도에 대해 부모가 아직 기다리고 있고 control flow가 그 Task 뒤로 진행하지 않은 경우만 허용한다. 같은 Task의 두 active attempt가 생기지 않도록 DB transaction에서 CAS한다. 자동 owner 반환이나 다른 역할로의 재할당은 없다.

7. Pi Adapter와 worker 프로토콜

7.1 채택 범위

Pi의 **agent-core + pi-ai**를 사용한다. coding-agent의 셸·파일 도구·프로젝트 확장 자동 로더는 사용하지 않는다. 현재 확인한 agent-core 1.2.0은 request 준비, 도구 전후 훅, 이벤트, 취소 연결점을 제공한다. 정확한 호출 순서는 잠근 버전의 소스로 contract test한다. [S06]

parent 서버는 authoritative 상태를, child worker는 하나의 Pi 실행 컨텍스트와 provider 연결을 가진다. child process는 정리·장애 격리용이며 신뢰할 수 없는 플러그인을 안전하게 실행하는 sandbox는 아니다. worker env는 필요한 provider 키와 런타임 설정만 전달한다. 실모델 연결은 최소 localhost 세션·Origin/Host 검사를 먼저 갖춘 상태에서만 활성화하며, P8까지 인증 없는 공개 live endpoint를 임시 운영하지 않는다.

7.2 Corral 포트

```
interface AgentAdapter {
  start(input: AgentStartInput, host: AgentHost): Promise<AgentHandle>;
}
interface AgentHandle {
  requestPause(): Promise<void>; // 경계 도달은 별도 observation
  cancel(reason: string): Promise<void>;
  waitForSettlement(): Promise<Settlement>;
}
interface AgentHost {
  prepareModelRequest(input: RequestProposal): Promise<RequestPermit>;
  invokeTool(input: ToolProposal): Promise<ToolOutcome>;
  observe(event: AgentObservation): Promise<void>;
}
```

지시 큐는 worker 내부의 유일한 비가시 상태로 두지 않는다. Coordinator가 directive 상태를 보관하고 request permit을 발급할 때 실제 요청 snapshot을 확정한다. Pi의 steering hook은 이 확정된 지시를 전달하는 수단이지 command 성공의 근거 자체가 아니다.

7.3 요청 생명주기

`request.proposed → admission/budget/context snapshot → worker dispatch → response stream → finalized usage/message → tool proposals → next request`.

도구 호출 순서는 첫 MVP에서 인스턴스 안에서는 sequential로 고정한다. 여러 child worker는 병렬 실행한다. 이 선택으로 지시 전달·대기·취소의 경계를 단순화한다. 병렬 tool batch의 전체 구현은 후속 작업으로 둔다.

`beforeToolCall`보다 `tool_execution_start`가 먼저 올 수 있으므로 Pi start를 곧바로 `tool.executing`으로 변환하지 않는다. `tool.requested`, `tool.allowed|blocked`, `tool.started`, `tool.finished`를 Corral에서 따로 정의한다. 차단된 도구에 실행 성공 기록이 생기지 않는 것이 P0의 필수 계약 테스트다. [S06]

7.4 IPC

worker→host는 `prepare_request`, `invoke_tool`, `observe`, `settled`; host→worker는 `permit`, `tool_result`, `pause`, `cancel`을 제공한다. 각 프레임에 protocolVersion, executionId, requestId를 넣고 schema 검증한다. 실행자의 instanceId나 권한을 모델이 보내는 인자로 신뢰하지 않는다. 해당 worker 채널의 바인딩으로 결정한다.

stderr와 stdout을 프로토콜로 혼용하지 않는다. 구조화 IPC 채널을 쓰고 diagnostic log는 별도로 수집·마스킹한다. worker의 예기치 않은 exit는 실패 관측이며 active Task를 조용히 completed로 바꾸지 않는다.

7.5 취소와 강제 종료

cancel은 먼저 Coordinator의 admission fence를 기록하고, worker에 Abort를 전달하며 backend tool fetch도 abort한다. 10초 안에 정리되지 않은 worker는 종료시킨다. child의 child를 worker가 직접 만들지 않고 모든 spawn을 host가 수행하므로 후손 registry가 완전해야 한다.

worker가 killed라도 원격 서비스의 실제 처리 완료 여부는 모를 수 있다. usage를 unknown으로 남기고 명확한 오류 상태를 제공한다. Pi 전체 API와 결합이 지나치게 커지면 adapter contract를 유지한 채 더 낮은 pi-ai 경계로 내리는 ADR을 열 수 있지만, 자동으로 자체 LLM loop를 추가하지 않는다.

8. 스케줄링·협업 프로토콜

8.1 슬롯

모델 호출 semaphore와 tool 호출 semaphore를 구분한다. 준비 대기 중 Task나 child 종료를 기다리는 parent가 모델 슬롯을 잡지 않는다. 예산 예약은 슬롯 admission과 같은 직렬 경로에서 처리하되, 오랫동안 semaphore를 기다리는 작업에 무제한 예산을 미리 묶지 않는다.

resident worker는 총 활성/대기 instance 12개로 제한한다. 동적 spawn/ask는 자식 worker 용량을 먼저 원자적으로 예약하고, 부족하면 전체 요청을 `RESOURCE_CAPACITY`로 거절한다. 부모를 먼저 waiting으로 바꾼 뒤 worker를 무기한 queue에 넣지 않는다. 모델 호출 permit 대기는 FIFO queue로 처리해도 되지만 resident process 용량 대기와 혼동하지 않는다. spawn batch는 용량 예약 전 일부만 생성하지 않는다. 기다리는 부모가 process 슬롯을 채워 자식을 시작할 수 없게 되는 반례를 테스트한다.

8.2 프로토콜 도구

도구	입력 요약	서버 측 효과
`spawn_subagents`	targetNodeId + task brief 목록	허용 link·depth·총량 검사 후 child Task/Execution 생성
`await_tasks`	소유 child taskId 목록	all 완료 또는 구조화된 실패/취소 결과
`ask`	허용 targetNodeId, body, 관련 버전	Question + response Task 생성 후 답변 대기
`reply`	questionId, body, evidence	열린 질문/담당 검증 후 단회 응답
`read_artifact`	scope 안의 artifactId/version	해석된 버전과 read event
`submit_result`	허용 artifactVersionIds	입력·출력 계약 검증 후 공식 출력 확정

일반 도구는 `search`, `read_document`, `write_artifact`. 역할에 필요한 protocol tool만 노출한다. 모델은 runId/owner/depth/budget 값을 생성하지 않는다. 서버가 context에서 채운다.

8.3 질문 응답

질문 한 번당 새 응답 Task를 생성한다. responder에게 전체 팀 대화가 아니라 질문·명시한 근거·해당 Role prompt만 전달한다. 질문자의 자식 실행으로 기록하며 callable target node의 설정을 적용한다. 이미 끝난 researcher 세션을 재사용하지 않는 이유는 컨텍스트·취소 소유권을 단순화하기 위해서다.

`reply`가 먼저, cancel이 나중에 확정되면 답변 결과는 보존하되 이후 Task 상태 정책을 적용한다. cancel/timeout이 먼저 확정되면 늦은 reply는 `message.reply_ignored` 감사 기록으로만 남긴다. 동일 답변을 두 번 submit해도 질문자가 두 번 깨어나지 않는다.

8.4 진행 없음

새 메시지 자체는 진전이 아니다. 완료한 child task, 새 제출 산출물, 승인 해결, 검토 기준의 충족 변화를 관측한다. 동일 내용 hash의 연속 산출물은 경고다. 자동 판단이 잘못될 수 있으므로 no_progress는 강제 성공/실패보다 `awaiting_user`로 보내고, wall deadline과 호출 hard cap은 별도로 유지한다.

9. Context·산출물·검토

9.1 ContextSnapshot

snapshot은 `executionId, modelCallId, rolePromptHash, configRevision, criteriaVersionId, orderedDirectiveIds, orderedMessageIds, resolvedArtifactRefs, modelProfileResolution, toolsHash, inputSummaryHash`를 포함한다. 실제 요청 메시지는 로컬 private 기록으로 보관하되 기본 공개 export에는 포함하지 않는다.

directive 내용은 snapshot에서 역추적 가능해야 한다. 스냅샷은 새 모델 호출마다 불변으로 만든다. provider가 입력을 처리했다는 확정이 없으면 delivered/unknown 구분을 남긴다. 모델이 실제로 지시를 지켰다는 증거는 별도의 평가다.

9.2 버전 바인딩 규칙

binding	해석 시점	예시
run_input	Run 시작	사용자가 입력한 brief
task_output	지정 선행 작업의 제출 완료	reviewer가 이번 writer의 draft를 읽음
previous_iteration_output	다음 회차 시작	writer가 직전 draft/review를 읽음
explicit_version	명령/정의에서 지정	검토 대상 draft@1
question_answer	답변 확정	질문자의 다음 컨텍스트

반복 시작의 pin은 **이전 회차에서 넘어오는 값**에만 적용한다. 이번 회차 writer의 새 출력은 reviewer 시작 시 task_output으로 별도 해석한다. 전역 latest에 의존하지 않는다. 허용된 API에서 latest를 받더라도 조회 시점에 실제 버전으로 고정해 read event에 남긴다.

9.3 제출·완료·사용자 승인

`write_artifact`가 만들어낸 초안은 바로 control flow에 노출하지 않는다. `submit_result`가 출력 계약을 통과해야 Task의 공식 output이 된다. reviewer Task의 제출이 유효한 review schema여도 그 verdict가 pass인지 별도로 판단한다.

Final Approval은 draftVersionId + reviewVersionId + criteriaVersionId를 묶은 대상을 가진다. 기준 patch는 Loop가 아직 종료되지 않은 상태에서만 허용하며 다음 회차 기준으로 적용한다. pending 기준 변경이 있으면 옛 기준 pass로 Loop를 종료하지 않고 다음 회차 진입 확인으로 이동한다. 이미 Loop가 종료되어 Final Approval에 도달했다면 기준 변경은 `CRITERIA_CHANGE_REQUIRES_NEW_RUN`으로 거절한다. 완료한 그래프 구간을 몰래 되돌리지 않는다. Loop 내부의 승인 요청은 기준 변경 시 stale로 바꾸고 새 대상 snapshot으로 다시 만든다. UI에서 '승인'만 눌러 새 버전이 자동 포함되지 않는다.

10. 예산·patch·경합

10.1 예약 장부

예산은 `spentKnown + reserved + spentUnknown`으로 구분한다. 요청 permit 발급 시 perCallMaxOutput을 예약하고, 공급자가 usage를 반환하면 실제 사용으로 정산한다. 미확인 취소는 보수적으로 예약분을 unknown에 이동한다. 나중에 같은 modelCallId의 usage가 확인되면 한 번만 정산한다.

환급과 추가 사용 반영은 같은 트랜잭션에서 처리한다. modelCallFinished 이벤트 재전송이 비용을 두 번 더하지 않는다. input/cached/reasoning tokens는 공급자 의미가 다르므로 rawUsage와 normalizedUsage를 함께 저장한다. 모든 토큰 수를 무조건 합산해 청구 토큰이라고 부르지 않는다.

금액 예약은 보조 정책이다. rate card가 없으면 금액은 null과 이유를 반환한다. 출력 토큰 한도만 확실히 지원되는 프로필이라면 그 한도와 모델 요청 수로 통제하고 금액 hard guarantee를 주장하지 않는다.

10.2 패치 충돌

패치에는 `expectedControlRevision`, `scope`, `effectiveBoundary`, `changes`를 포함한다. 같은 node의 prompt를 동시에 수정한 두 명령은 하나만 먼저 확정하고 나머지는 conflict로 돌려준다. 단일 사용자라도 탭 두 개에서 충돌할 수 있다.

queued execution의 설정 freeze는 `execution.activated` 시점이다. 생성되었어도 아직 활성화하지 않은 미래 실행은 patch 적용 대상이다. active execution은 모델/role config가 고정되며 새 자연어 directive만 다음 request로 받을 수 있다.

10.3 경합 표

경합	먼저 확정된 사실	다음 처리
steer vs request prepare	snapshot 먼저	지시는 그다음 요청 대상
steer vs execution finish	finish 먼저	TARGET_TERMINAL, 새 작업/다음 회차 안내
cancel vs tool completion	cancel 먼저	감사·usage만 허용, Task 부활 금지
reply vs question timeout	timeout 먼저	late reply ignored
approve vs criteria patch	patch 먼저	STALE_APPROVAL
budget decrease vs admission	admission 먼저	이미 예약한 요청 유지/명시 cancel. 사용량 초기화 금지
restart vs 늦은 이전 결과	restart epoch 먼저	이전 epoch 결과는 감사만, 새 시도 오염 금지
D&D template edit vs live Run	별개 definition revision	Run base snapshot 불변

11. SQLite 데이터 모델

DB는 로컬 디스크에 둔다. 네트워크 파일시스템의 WAL 지원을 가정하지 않는다. `foreign_keys=ON`, WAL, 적절한 busy timeout, 강한 동기화 설정을 명시하고 실제 driver 테스트를 한다. SQLite의 트랜잭션/WAL 기능을 사용하되, 외부 모델 호출까지 DB 원자성이 확장된다고 해석하지 않는다. [S20-S22]

테이블	핵심 필드·제약
corrals	id, name, current_revision
definition_versions	corral_id + revision UNIQUE, definition_hash, canonical_json
layouts	corral_id + layout_revision, definition_revision, layout_json
documents	id, source, content_hash, text_body, imported_at, owner_scope
runs	id, base_definition_hash, snapshot_json, status, control_revision, last_seq, process_epoch
commands	run_id + command_id UNIQUE, payload_hash, receipt_json
run_events	run_id + seq UNIQUE, event_id UNIQUE, type, entity refs, cause_ids_json, payload_json
executions	id, run_id, node_id, role_id, task_id, attempt, parent_id, epoch, config_revision, status
tasks	id, run_id, assigned_role_id, active_execution_id, status, output_refs_json, revision

테이블	핵심 필드·제약
messages	id, type, task_id, sender_id, target_id, question_id, body, status
artifact_versions	artifact_id + version UNIQUE, task_id, author_id, content_hash, text_body, lineage_json
directives	id, target_scope_json, status, content, receipt_seq, delivery_model_call_id
run_patches	id, run_id, control_revision UNIQUE per run, before_after_json, effective_scope
approvals	id, target_snapshot_hash, status, resolved_by, resolution_command_id UNIQUE
model_calls	id, execution_id, permit_id UNIQUE, context_snapshot_id, lifecycle, raw_usage_json
budget_ledger	reservation_id UNIQUE, model_call_id, amounts, status
effect_intents	effect_id UNIQUE, owner_scope, epoch, kind, status, payload_ref
context_snapshots	id, execution_id, model_call_id UNIQUE, canonical_json, hash

외래키와 UNIQUE로 가능한 불변식을 DB에서도 강제한다. 변경 가능한 현재 상태 테이블은 events로 재구성 가능해야 한다. 스키마 마이그레이션은 버전별로 검증하며, 실제 사용자 DB에 `push`로 즉흥적인 구조 변경을 하지 않는다.

12. HTTP·SSE 계약

12.1 API 표

Method / path	입력·결과	제약
GET `/api/v1/health`	버전·기능·process epoch	secret 없음
POST `/api/v1/session/bootstrap`	일회성 local bootstrap token → 세션	localhost/Origin 검증
GET/POST `/api/v1/corrals`	목록/새 정의	schema validation
GET/PUT `/api/v1/corrals/:id`	정의 저장·불러오기	If-Match definition revision
PUT `/api/v1/corrals/:id/layout`	레이아웃 저장	독립 layout revision
POST `/api/v1/corrals/validate`	정의 → 오류·실행 계획 요약	실행 효과 없음
POST `/api/v1/runs`	corralId + definitionRevision + 입력 + mode + commandId	run 생성 멱등성·한 개 active 제한
GET `/api/v1/runs/:id/snapshot`	projection + lastSeq	일관된 DB snapshot

Method / path	입력·결과	제약
GET `/api/v1/runs/:id/events?after=N`	SSE 또는 JSON page	커서·순서 보장
POST `/api/v1/runs/:id/commands`	typed command → receipt	commandId 필수
GET `/api/v1/runs/:id/commands/:commandId`	결과 재조회	응답 유실 복구
GET `/api/v1/runs/:id/artifacts/:id/versions/:n`	immutable body+metadata	범위·접근 확인
POST `/api/v1/documents/import`	text/Markdown/허용 URL	size·URL 정책
POST `/api/v1/runs/:id/export`	redacted archive	private transcript는 별도 opt-in

HTTP 202는 명령을 접수했다는 뜻일 뿐 적용 완료가 아니다. receipt에는 `acceptedSeq, status, target, requestedBoundary, appliedRevision?, deliveryIds?, rejection?`를 반환한다. UI는 확정 event 또는 receipt 조회로 상태를 갱신한다.

실행 제어에 optimistic success UI를 사용하지 않는다. 버튼은 전송 직후 pending으로 바뀌되, 실제 paused/cancelled 배지는 서버의 확정 사실을 따른다.

12.2 SSE 재연결

영속 domain event에는 SSE `id`로 runSeq를 사용한다. snapshot을 읽어 watermark N을 받고 `after=N`으로 구독한다. replay 쿼리와 live 구독 사이 누락이 없도록 서버가 구독 버퍼를 만든 뒤 high watermark까지 DB에서 전송하고 버퍼를 중복 제거한다.

UI는 중복 seq를 무시하고 기대 seq에 gap이 있으면 복구 쿼리를 한다. 알 수 없는 schemaVersion을 받으면 표시를 중지하고 갱신 필요를 알린다. snapshot의 seq보다 오래된 이벤트를 적용하지 않는다.

텍스트 delta는 별도의 비영속 `stream_frame`이며 modelCallId+chunkIndex로 구분한다. 기본 replay의 기준은 최종 메시지 스냅샷이다. 모든 토큰을 SQLite 이벤트로 저장하지 않는다. 스트림 gap에는 최종 메시지 조회로 복원하고, 근거 없는 텍스트를 이어 붙이지 않는다.

slow client queue가 상한을 넘으면 무제한 메모리 축적 대신 연결을 끊고 재동기화를 요청한다. domain event는 DB에서 다시 읽을 수 있고, delta는 최종 텍스트로 복구한다.

13. 프런트엔드 상태·상호작용

13.1 데이터 소유

Zustand의 editor store는 draft definition/layout, 실행 projection store는 정규화된 read model, Reins store는 form/pending command, stream store는 진행 중 텍스트를 가진다. TanStack Query는 정의 목록·과거 기록·문서 상세 같은 요청 캐시를 맡는다. 동일한 live Run의 진실을 Query와 Zustand 두 곳에서 독립적으로 변경하지 않는다.

React Flow controlled nodes/edges는 definition+layout의 selector 결과다. drag 이벤트는 위치 store를 갱신하고 드래그 끝에 persist한다. model delta가 nodes 배열을 통째로 새로 만들지 않는다. Run overlay는 nodeId별 작은 selector를 구독한다. [S02, S08]

13.2 컴포넌트

`CorralCanvas`, `NodePalette`, `AgentNodeView`, `DecisionNodeView`, `LoopGroupView`, `OutputNodeView`, `ConnectionInspector`, `ValidationPanel`, `ExecutionTree`, `WaitingReasonLink`, `TaskBoard`, `MessageThread`, `ArtifactDiff`, `ReinsPanel`, `DirectiveReceipt`, `BudgetInspector`, `ApprovalCard`를 분리한다.

처음부터 공개 UI 라이브러리가 되는 것은 아니지만 `packages/react`에 제품용 컴포넌트와 Storybook story를 둔다. 노드의 안/밖 선택 상태, 길어진 이름, 대기/취소/오류, 좁은 폭, keyboard focus를 각각 story로 만든다.

13.3 렌더링·레이아웃

Canvas를 대형 채팅 메시지 렌더링과 분리한다. Timeline/실행 트리는 TanStack Virtual을 사용해 가상화하고, 전체 트리 flatten은 memoized selector로 계산한다. 이벤트는 16~50ms 단위로 화면 반영을 모을 수 있지만 영속 순서와 제어 receipt를 누락하지 않는다.

자동 레이아웃은 초기 MVP 필수가 아니다. 사용자가 정한 위치를 실행 이벤트 때문에 재배치하지 않는다. 필요하면 후속 ELK adapter로 확장하되 graph compiler와 결합하지 않는다. React Flow Pro 전용 예제 코드를 제품의 필수 의존성으로 두지 않는다. Undo/Redo·멀티선택은 공개 API와 자체 command history로 구현한다.

13.4 접근성

드래그 이외에 `Add node` 메뉴와 source/target 선택 방식의 연결 대안을 둔다. 키보드 focus와 그래프 선택을 구별하며 Reins에서 Escape 시 원래 실행 항목으로 복귀한다. 상태 설명은 aria-live에 매 token마다 읽히지 않게 중요 전이만 알린다.

IME composing 동안 Enter는 전송·저장·노드 단축키를 실행하지 않는다. 색과 애니메이션은 보조이며 prefers-reduced-motion에서 흐르는 선 애니메이션을 끈다. 자동 접근성 검사만으로 접근성을 모두 보장한다고 보고하지 않는다. [S03, S18]

14. 보안 경계와 자료 읽기

모델 provider와 검색 API는 고정된 설정의 endpoint만 사용한다. 사용자 문서 본문이나 모델 응답에서 endpoint/key를 바꾸게 하지 않는다. model profile은 server side capability registry로 해석하고 client export에는 secret ref도 최소화한다.

local bootstrap token은 최초 연결에서만 사용하고 로그/URL query에 장기 secret을 남기지 않는다. UI·API same-origin을 기본으로 하며 CORS wildcard는 금지한다. state-changing request는 session+CSRF+Origin 검증을 수행한다. DNS rebinding을 고려해 Host allowlist를 둔다.

외부 문서 fetch는 HTTPS, URL canonicalization, 모든 A/AAAA 주소 검사, private/link-local/loopback/metadata 차단, 리다이렉트 hop마다 재검사, 실제 연결 주소 pinning, 응답 크기·압축 해제 크기·time limit, 허용 MIME을 적용한다. 사용자가 승인한 URL이어도 네트워크 안전 검증을 건너뛰지 않는다. [S24]

MVP 자료는 등록한 text/Markdown와 안전하게 추출한 HTML text다. PDF·Office parser·OCR는 제품 runtime에는 넣지 않는다. source import 과정에서 script·외부 리소스를 실행하지 않는다. jsdom/Readability를 사용하면 resource loading과 script execution을 켜지 않으며 파서 기능이 보안 sandbox를 대체하지 않는다.

기본 검색은 mock corpus에 동작한다. 실제 검색은 SearchProvider 뒤에 Brave Search API 하나를 연결한다. 이는 OSS 라이브러리와 별개인 외부 서비스이며 사용자 키·이용 비용이 필요할 수 있다. 서비스 가격·무료량을 제품에 하드코딩하지 않는다. 추후 self-hosted 검색 adapter를 붙일 수 있지만 MVP에 새 검색 서버 설치를 강제하지 않는다. [S25]

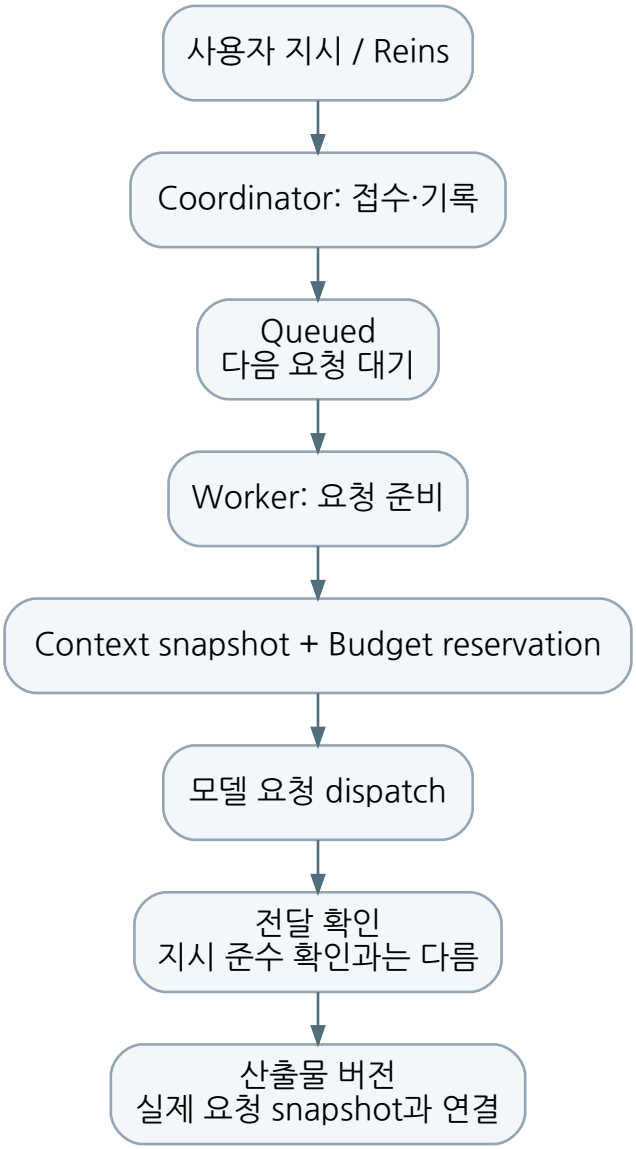
15. 확장 포트와 보류 기준

`DecisionProvider`는 이번 단계에서 플러그인 시스템으로 만들지 않고, DecisionNode에서 결과를 받는 작은 내부 포트만 유지한다. v0.2에 동일 질문 계약의 `Local/Jev` adapter를 넣는다. Node.js↔Python 실행 경계를 지금부터 강제하지 않는다.

`GraphRunner`와 `AgentAdapter`는 분리한다. LangGraph가 맡는 순회와 Pi가 맡는 모델/도구 세션을 변경해도 public command/event contract가 변하지 않게 한다. 다만 adapter 교체만으로 모든 프레임워크 내부를 관측할 수 있다고 약속하지 않는다.

기술 스파이크 실패 시 순서는 **계약 재확인 → 해당 adapter만 수정 → ADR로 기반 변경 검토**다. 상태 계약을 약화하거나 D&D를 제외하는 것은 해결책이 아니다.

부록 도식: 스티어링의 관측 경계



접수·요청 전달·산출물 생성을 구별한다. Mermaid sequenceDiagram 원본: diagrams/reins.mmd

PART 03

API·이벤트 계약

명령 · Receipt · 오류 · SSE · 이벤트 payload

1. 명령과 조회

모든 주소는 제안된 로컬 same-origin `/api/v1` 기준이다. 인증 세션, workspace binding, Host/Origin 검사 후 처리한다. API key는 이 API의 응답에 포함하지 않는다.

Method/path	요청	응답·효과
GET /corrals	cursor?	정의 목록; 본문·레이아웃은 필요 시 조회
POST /corrals	정의+레이아웃	새 저장 정의. clone 시 ID 재발급
GET /corrals/:id	—	definition/layout/revisions
PUT /corrals/:id	If-Match definition revision,definition	conflict 검사 후 저장; 실행 중 Run 원본에는 영향 없음
POST /corrals/validate	현재 draft definition	diagnostics + executable IR summary; side effect 없음
POST /runs	commandId,corralId,definitionRevision,input,mode	runId+receipt; validation 완료 전 모델 호출 없음
GET /runs/:id/snapshot	—	read model + snapshotSeq
GET /runs/:id/events?after=N	cursor	commit된 domain events의 SSE; token frames 별도 type
POST /runs/:id/commands	command.schema.json	accepted receipt 또는 4xx; HTTP202는 적용 완료가 아님
GET /runs/:id/commands/:commandId	—	command 상태·관련 events·적용 scope
GET /runs/:id/artifacts/:artifactId/versions/:version	—	불변 본문·생성/읽기/지시/검토 경로
GET /model-profiles	—	public profileId·지원 capability·keyConfigured, secret 없음
POST /runs/:id/export	redaction policy	공개용 bundle. 원문 비공개가 기본

SSE event는 `id: runSeq`, `event: domain`을 가진다. SnapshotSeq 이하 event는 재적용하지 않는다. Last-Event-ID와 after가 충돌하면 서버 계약으로 하나를 선택하고 오류/우선순위를 문서화한다. 첫 구현은 Last-Event-ID가 유효하면 우선한다. 범위 밖 cursor·다른 Run cursor는 400이다.

2. Receipt

receipt는 commandId, runId, status, acceptedSeq, target, requestedBoundary, relatedEventIds, error를 갖는다. status는 `accepted / queued / applied / rejected / superseded / expired / cancelled`다. 일부 target에만 적용하는 다중 대상 steer는 MVP에 제공하지 않아 부분 적용 모델을 미룬다.

steer receipt의 applied는 입력 dispatch 확인을 의미한다. 품질 준수 여부는 review로 별도 표시한다. 설정 patch의 applied는 policy commit이며, 미래 execution이 실제 사용한 patch.activated와 구분한다.

3. 오류

코드	HTTP	의미
SCHEMA_INVALID	400	이해하지 못하는 형식·unknown field
GRAPH_INVALID	422	포트·Loop·role·binding 등 의미 불일치
CAPABILITY_DENIED	403	도구/질문/위임 관계가 허용되지 않음
STALE_CONTROL_REVISION	409	준비 중 다른 제어 변경이 발생
IDEMPOTENCY_CONFLICT	409	같은 commandId에 다른 payload
TARGET_TERMINAL	409	종료된 실행에 지시 등 적용 불가
RESOURCE_CAPACITY	409	동적 자식/응답 worker 용량 예약 실패. 부모를 무기한 대기에 넣지 않음
CRITERIA_CHANGE_REQUIRES_NEW_RUN	409	Loop가 이미 종료되어 완료 기준 변경은 새 Run 필요
RESTART_NOT_ALLOWED	409	부모가 더 이상 대기하지 않거나 downstream 진행
APPROVAL_STALE	409	대상/기준 버전 변경 또는 이미 해결
LIMIT_REACHED	409	새 effect admission 불가; 별도 approval/상한 변경 필요
STORAGE_UNAVAILABLE	503	사실 기록이 불가하므로 신규 admission 닫음
SESSION_INVALID	401	세션/부트스트랩 오류
ORIGIN_REJECTED	403	허용하지 않는 Origin/Host/CSRF

provider/network error는 사용자 키 원문·응답 헤더를 포함하지 않는다. trace용 errorId와 마스킹된 이유를 제공한다. 도구 실행 중 실패는 HTTP success/error와 별개로 ToolOutcome에 structured error를 넣는다.

4. 이벤트 payload 목록

봉투 schema만 이 팩에 포함되어 있다. 아래 payload별 검증은 C-005에서 Zod discriminated union으로 구현하고 fixture를 늘린다. causal graph는 cause 1개에 한정하지 않는다.

type	반드시 기록할 내용
`run.started`	정의/입력/모델 snapshot과 최초 limits
`run.state_changed`	from,to,reasonCode
`run.interrupted`	previousStatus,backendSessionId
`execution.created`	parentExecutionId,nodeId,taskId,attemptNo,depth,iterationId
`execution.state_changed`	from,to,activity,waitingOn
`execution.settled`	outcome,terminationMode,error
`task.created`	role,node,contract,dependencies
`task.attempt_assigned`	executionId,attemptNo
`task.submitted`	artifactVersionIds
`task.completed`	completionEvidence
`task.blocked`	cause,requiredUserAction
`message.question_created`	questionId,taskId,sender,targetRole,body,refs,deadline
`message.answer_committed`	questionId,responseTaskId,body,refs
`message.closed`	questionId,answer
`artifact.version_created`	artifactId,version,bodyRef,hash,parentVersion,authorExecutionId,modelCallId,contextSnapshotId
`artifact.read`	requestedRef,resolvedVersionId,consumerExecutionId
`review.recorded`	reviewVersion,targetVersion,criteriaVersion,verdict,issues
`review.stale`	reviewVersion,newCriteriaVersion,cause
`loop.iteration_started`	iterationId,no,pinnedCarryInputs,criteriaVersion
`loop.iteration_finished`	outcome,submittedVersions
`approval.requested`	approvalId,kind,subjectVersions,criteriaVersion
`approval.resolved`	decision,actor,subjectVersions
`directive.accepted`	directiveId,target,body,commandId
`directive.attached`	contextSnapshotId,modelCallId
`directive.dispatched`	modelCallId,executionEpoch
`directive.undelivered`	reason
`patch.committed`	patchId,scope,before,after,controlRevision
`patch.activated`	patchId,executionId

type	반드시 기록할 내용
`effect.admitted`	effectId,scope,epoch,reservationId
`effect.dispatch_observed`	effectId,providerRequestId
`effect.settled`	effectId,outcome
`tool.requested`	toolCallId,name,validatedArgs
`tool.allowed`	toolCallId,policyVersion
`tool.blocked`	toolCallId,reason
`tool.started`	effectId,toolCallId
`tool.finished`	resultRef,error
`model.request_prepared`	modelCallId,contextSnapshotId,profile,configRevision
`model.finished`	responseRef,rawUsage,normalizedUsage,certainty
`budget.reserved`	reservationId,amount,scope
`budget.settled`	reservationId,actual,unknownOutstanding
`limit.reached`	limitKind,used,reserved,max
`cancel.requested`	scope,fenceSeq,reason
`cancel.confirmed`	scope,settlementRefs
`observation.late`	originalEvent,whyIgnored
`output.finalized`	artifactVersionIds,approvalId

5. timeout·재연결·다중 탭

브라우저 timeout은 서버 명령 실패와 같지 않다. 같은 commandId로 receipt를 조회/재전송한다. 새 ID로 취소/승인을 재시도하면 이중 의미가 생길 수 있으므로 client SDK가 처리한다.

workspace의 활성 Run은 하나지만 탭은 여러 개일 수 있다. 탭별 controlRevision 갱신과 conflict UI가 필요하다. SSE가 끊겼다고 Run을 자동 취소하지 않는다. Run-level wall deadline과 승인 대기 상태를 유지한다.

PART 04

세부 구현 플랜

SDD / TDD · P0-P9 · 50개 태스크 · 일정 · 완료 게이트

> 기준일: 2026-10-14. 제품 범위의 기준은 Corral-Spec-v0.1.md다. 이 플랜은 구현 전 계획이며, 명령 예시는 앞으로 저장소에 만들 스크립트를 뜻한다. 실행 가능한 완성 애플리케이션을 제공했다는 의미가 아니다.

1. 구현 원칙

첫 사용자에게 보이는 MVP부터 D&D·협업·Reins가 함께 있어야 한다. 기술 스파이크가 내부적으로 먼저 존재할 수는 있지만, 실행기만 있는 결과물을 MVP라고 부르거나 D&D를 후속 범위로 이동하지 않는다.

개발을 backend 전부→viewer→editor 순서로 진행하지 않는다. 첫 단계부터 Canvas와 Reins를 mock으로 동작시키고, 각 파트가 계약을 공유한 상태에서 실제 runtime으로 바꾼다. frontend는 동작이 없는 시안이 아니라 매 단계에서 실행·테스트 가능한 세로 절편을 제공한다.

작은 PR의 기본 단위는 **요구사항 ID + 실패하는 검증 + 최소 구현 + 문서/테스트 증거**다. 라이브러리 도입과 기능 구현을 구분하고, 의존성을 설치했다는 이유로 해당 기능을 완료로 판정하지 않는다.

2. 채택 기술과 배제 결정

계층	채택	이유	지금 추가하지 않을 것
앱	React + TypeScript + Vite	로컬 도구·정적 데모·Storybook에 같은 UI 사용	Next.js 서버 기능, 별도 BFF
그래프 UI	React Flow (`@xyflow/react`)	노드·포트·선택·뷰포트 기반	다른 D&D 엔진 중복, 유료 Pro 예제 필수화
상태	Zustand + TanStack Query	편집/live projection과 일반 query cache의 역할 분리	Redux/XState 동시 채택
UI 기초	Radix Primitives + Tailwind CSS + CSS tokens	접근 가능한 primitive에 자체 디자인 적용	테마 은유를 위해 전체 UI kit 이중 사용
폼·본문	React Hook Form, react-markdown/remark-gfm, diff	프롬프트 입력, 안전한 Markdown, 버전 비교	Monaco/전체 문서 편집기
가상화	TanStack Virtual	Timeline·실행 트리의 큰 목록 처리	매 이벤트 전체 DOM 생성
API	Fastify + typed REST + SSE	명령/receipt와 이벤트 읽기를 분리	GraphQL/Socket.IO 필수화
계약	Zod 4 + generated JSON Schema	서버·클라이언트·fixture의 같은 검증	런타임에서 임의 코드로 schema 생성

계층	채택	이유	지금 추가하지 않을 것
workflow	LangGraph.js 내부 adapter	제한된 graph 순회·조건 분기 기반	LangGraph/Pi/DB의 독립 상태 원본 3개
Agent	Pi agent-core + pi-ai	범용 역할·provider·tool loop 재사용	coding-agent 기본 셸/프로젝트 확장 로딩
저장	SQLite + better-sqlite3 + Drizzle	로컬 단일 파일, transaction·typed SQL	Postgres/Redis/S3 필수 인프라
검증	Vitest + Testing Library + Playwright + MSW + fast-check	단위/DOM/실브라우저/HTTP/model-based 검증 분리	real LLM을 매 PR 테스트의 유일한 판정자로 사용
UI 증거	Storybook, Playwright trace·screenshot	상태별 UX와 재현 자료	특정 유료 CI 서비스에 종속
품질/배포	pnpm workspace, Biome, GitHub Actions	한 TS 모노레포·잠근 의존성	첫 버전부터 npm 공개 패키지 여러 개
SDD 보조	Markdown·schema·ID 매핑, Spec Kit 선택 사용	명세를 Git과 CI에서 관리	특정 LLM 전용 워크플로 강제

유지보수 근거·라이선스·확인한 릴리스와 버전 정책은 Dependencies-and-Sources.md에 있다. exact version 잠금은 P0에서 Node/peer/type 호환성을 시험한 후 결정한다. 문서에 표시한 최신 관측 버전이 곧 통합 검증된 lockfile이라는 뜻은 아니다.

3. SDD 작업 흐름

3.1 스펙 계층

```
제품 헌장 / MVP 비협상 항목
    ↓
기능 spec: 사용자 동작, 상태·경계, 오류, 수용 기준
    ↓
계약: JSON schema, event/command, 상태 전이, API
    ↓
테스트 설계: normal / failure / race / security / a11y
    ↓
작업 분해: 담당 모듈, 의존 태스크, 증거
    ↓
RED → GREEN → REFACTOR
    ↓
통합 검증 / 문서 일치 확인 / 사람 리뷰
```

필수 feature 묶음은 `001-canvas`, `002-run-coordinator`, `003-agent-adapter`, `004-collaboration`, `005-review-loop`, `006-reins`, `007-limits-recovery`, `008-live-tools-security`, `009-demo-release`다. 각 feature는 `spec.md`, `plan.md`, `tasks.md`, `acceptance.feature`, `evidence/`를 가진다. 루트 문서를 복붙하지 않고 필요한 요구 ID를 참조한다.

3.2 착수 가능 조건(Definition of Ready)

작업은 requirement ID, 정상/거절 경로, 입력·출력 예제, 적용 경계, 변경 소유 모듈, 의존 작업, 검증 명령이 있어야 시작할 수 있다. 동시에 바뀌는 두 계약의 순서가 불명확하면 코드부터 만들지 않고 Change Request를 작성한다.

명세 변경은 `why / affected requirements / compatibility / tests / migration / owner decision`을 기록한다. “MVP라 생략”이라는 설명만으로 필수 요구를 제거할 수 없다. assistant나 구현 에이전트가 임의로 수용 기준을 낮추지 않도록 저장소의 AGENTS 지침에 고정한다.

3.3 계약의 기준 파일

미래 저장소에서는 `packages/contracts/src/`의 Zod가 wire schema의 기준이며 JSON Schema/OpenAPI는 생성물로 관리한다. prose는 사용자 의미와 상태 규칙의 기준이다. 둘이 다르면 테스트와 spec review로 수정한다. 이 문서 패키지의 JSON Schema는 **설계 시점의 seed**이며, 구현 시 schema source로 옮긴 뒤 같은 fixture에 대한 parity 테스트를 통과시켜야 한다.

`requirements.json → test-catalog.json → backlog.json`을 CI에서 검사한다. 요구 없는 구현 태스크, 테스트 없는 필수 요구, 정의되지 않은 ID를 실패로 처리한다. 이 패키지는 해당 매핑의 출발 데이터를 포함한다.

4. TDD 규칙

RED에서는 의도한 assertion이 실패하는지 확인한다. import 누락·환경 파손으로 실행이 안 되는 것을 기능의 RED 증거로 대체하지 않는다. GREEN은 그 동작을 통과시키는 최소 구현이다. REFACTOR 후 동일 테스트와 인접 계약을 다시 실행한다.

핵심 runtime 로직은 Clock, IdFactory, ModelAdapter, SearchProvider, Storage, EffectDispatcher를 주입 가능하게 작성한다. 테스트에 실제 시간 지연·실모델 확률·외부 웹 문서 상태를 넣지 않는다. 취소/응답 경합은 barrier가 있는 fake adapter로 호출 순서를 직접 제어한다.

실제 LLM 테스트는 `live` suite로 분리하고 예산·timeout·opt-in key를 요구한다. 기록에는 provider/model의 실제 ID, 시각, input fixture hash, API 설정을 남긴다. 모델이 한 번 성공했다는 결과를 deterministic 계약의 증거로 사용하지 않는다.

coverage는 보조 신호다. core/contracts는 초기 목표 line 90% / branch 85%, 나머지는 중요한 실패 경로 중심으로 관리한다. 숫자는 계획 목표이며 달성 보고가 아니다. 추후 mutation test는 Reins·budget·cancel reducer부터 제한적으로 적용한다. 거대한 snapshot만으로 테스트를 채우지 않는다.

5. 단계·의존성·산출물

아래 시간은 **추측에 기반한 계획용 집중 작업 시간**이다. AI 도구 사용에 따른 생산성 배수를 검증하지 않았으므로 할인하지 않았다. 리뷰·통합 리스크를 포함한 최종 달력 일정은 P0/P1 실제 작업량을 측정한 뒤 보정한다.

단계	범위	시간 추정	완료 증거
P0	명세·의존성 잠금과 첫 Canvas 세로 절편	18-28h	동결할 계약, MockAdapter, 실제 드롭 가능한 화면을 동시에 만든다.
P1	D&D 편집기·정의·compiler	30-46h	지원 노드와 연결이 실제 실행 계획으로 변환된다.
P2	Coordinator·SQLite·실행 투영	26-40h	영속된 사실만 화면에 표시되고 명령 재전송이 안전하다.
P3	Pi worker와 모델 호출 계약	22-36h	모델 두 프로필과 요청·도구·취소·usage를 연결한다.

단계	범위	시간 추정	완료 증거
P4	작업·질문·서브에이전트 협업	30-48h	동적 자식과 메시지 왕복이 한 Run에서 연결된다.
P5	검토 루프·버전·승인	20-32h	이전 회차 입력과 현재 출력이 섞이지 않는다.
P6	Reins 스티어링 전체	32-50h	접수·전달·미래 설정·pause·개별 취소의 계약이 성립한다.
P7	예산·경합·장애·재열람	26-42h	폭주와 늦은 결과에 상태가 무너지지 않는다.
P8	실자료·보안·CLI	22-36h	안전한 검색/읽기, 로컬 인증, 동일 CLI 경로를 완성한다.
P9	접근성·성능·공개 데모·릴리스	24-42h	기능뿐 아니라 조작·관측 경험을 검증한다.

전체 기본 합계는 **250-400시간**이다. 불확실성 여유 20%를 더한 계획 버퍼는 300-480시간이다. 주 10시간이면 약 30-48주, 주 20시간이면 약 15-24주로 환산된다. 업무 병행·API 장애·UI 완성도에 따라 달라지며 납기 보장이 아니다. 기능 수를 절반으로 줄이는 방식이 아니라, 각 단계의 실제 측정으로 먼저 재산정한다.

P0 — 계약과 첫 화면을 함께 만든다

제품 헌장, Graph/Task/Execution/Directive 상태, package 경계, source/license record를 잠근다. React Flow의 실제 drop/connect와 mock Run 버튼, Reins 입력창을 만들고 story/E2E를 작성한다. 동시에 Pi request/tool hook·cancel·usage와 LangGraph callback 재진입을 최소 스파이크로 확인한다.

Exit gate: 첫 화면에서 Agent→Output을 만들고 모의 실행을 볼 수 있다. 지시는 pending으로 표시된다. backend 없는 viewer만으로 P0를 닫지 않는다. Pi 훅이 기대와 다르면 버전·source를 기록하고 adapter ADR을 결정한다.

P1 — 사용자가 정의를 만드는 경험

네 노드, Loop grouping, capability link, inspector, validation panel, 저장·불러오기, Undo/Redo를 만든다. compiler 오류에 nodeId/edgeId를 넣고 클릭으로 이동한다. layout 이동과 실행 의미 변경을 다르게 저장한다.

Exit gate: 템플릿뿐 아니라 빈 캔버스에서 만든 유효한 정의가 IR로 변환된다. 잘못된 연결은 실행 전에 거절되고 위치가 표시된다. 키보드 대안을 같은 기능 경로로 제공한다.

P2 — 단일 상태 확정과 실시간 투영

SQLite migration, command dedupe, decide/reduce, transactional events, effects, REST/SSE를 붙인다. LangGraph callback은 mock Agent를 Coordinator 경로로 실행한다. Canvas overlay, Run tree, 대기 이유 링크를 연결한다.

Exit gate: 응답 유실 후 command 재전송이 중복 Task를 만들지 않는다. snapshot+stream 간 race가 없다. read model을 이벤트로 재생성해 동일 결과를 얻는다.

P3 — 실제 Agent를 작은 계약으로 연결

Pi worker를 얇은 IPC adapter로 연결한다. 비밀 키 전달 범위, request snapshot, tool permission, abort를 구현한다. provider 프로필은 두 개로 제한하고 자동 모델 라우팅은 넣지 않는다. 텍스트 스트림은 state event와 분리한다.

Exit gate: 두 provider의 실제 호출·usage·cancel smoke 결과가 기록된다. blocked tool이 실행 이벤트로 표시되지 않는다. live smoke는 통합 test와 분리되어 있다.

P4 — 협업의 핵심을 연결

작업판, 자식 생성/합류, ask/reply, 불변 산출물, submit을 만든다. “기다리는 부모도 슬롯을 점유하는” 오류를 test-first로 막는다. responder는 새 Task이므로 권한·상한·인과·취소가 모두 동일 경로를 사용한다.

Exit gate: 조사자 두 자식 실행과 Writer 질문 왕복을 한 Run에서 관측할 수 있다. side chat가 아니라 task/message/artifact가 연결되어야 한다.

P5 — 검토와 기준의 정합성

현재 회차/이전 회차 바인딩, review schema, 수정 Task, cap gate, 최종 승인을 연결한다. ArtifactDiff에 원천 버전과 기준 버전을 표시한다.

Exit gate: reviewer가 새 draft를 읽고, old review를 새 draft 승인으로 재사용하지 않는다. 승인 중 기준 변경은 stale이다. mock 실패→통과 시나리오와 실제 LLM 평가 결과를 구별한다.

P6 — Reins를 제품 수준으로 완성

live directive, pause/drain/resume, partial/global cancel, 조건부 restart, future node config patch를 구현한다. 각 동작은 대상·경계·receipt·거절 이유를 가진다. 사용자 지시와 결과의 연결은 시각이 아니라 실제 snapshot에서 만든다.

Exit gate: 실행 중 입력한 지시가 다음 request에 들어간 사실을 UI로 확인한다. cancel 후 부모/질문자가 대기에 남지 않는다. 완료한 노드를 임의 재실행하지 않는다. 미래 patch가 template나 active config를 덮지 않는다.

P7 — 경합과 복구를 제품 계약으로 만든다

budget reservation, unknown usage, 상한 수정, wait/no-progress, cancel vs reply 경합, crash interrupted, export/reopen, SSE backpressure를 검증한다. fake clock과 randomized command trace의 seed를 보관한다.

Exit gate: 순서를 뒤섞어도 상태 불변식이 유지된다. storage 실패 때 새 외부 효과가 실행되지 않는다. backend 재시작 후 외부 호출이 재발행되지 않는다.

P8 — 실자료와 안전한 사용 경계

로컬 세션·Origin/Host 검증, 등록 자료 import, 안전한 HTTPS 읽기, optional live search, CLI를 연결한다. CLI는 REST 명령과 같은 Coordinator를 사용한다. 키 없는 테스트와 API 키가 필요한 기능을 명확히 나눈다.

Exit gate: malicious URL/redirect/HTML/import를 차단한다. inspect는 키 없이 작동하며 run은 승인 대기를 자동 우회하지 않는다.

P9 — 공개 가능한 MVP

다중 상태 story, 키보드·IME·a11y, event flood 성능, 정적 mock demo, CI·라이선스·SBOM·readme를 마무리한다. 데모에서 D&D/Reins 변경이 실제 mock 결과를 바꿔야 하며 단순 영상 재생이면 안 된다.

Exit gate: 모든 AC와 FR의 증거가 연결된다. live smoke와 local demo를 구별하고 known limitation을 공개한다. 후속 Local/Jev를 구현된 기능처럼 설명하지 않는다.

6. 작업 백로그

모든 항목은 planned 상태다. 완료 여부는 아래 done 조건과 테스트 증거로 갱신한다. 기계 판독 데이터는 backlog.json에 있다.

ID	단계	작업	의존	완료 조건
C-001	P0	제품 헌장·상태/개입 계약·ADR 동결	-	접수/전달/종료의 뜻을 정의하고 테스트 이름까지 매핑
C-002	P0	의존성 lock·OSS 근거·호환성 스파이크	C-001	정확한 버전과 Node24, React, Pi, Vitest peer 검사 기록
C-003	P0	MockClock·IdFactory·ScriptedAgent 만들기	C-001	네트워크 없이 동일 입력/seed에 동일 확정 이벤트
C-004	P0	React Flow 첫 Canvas+Reinsstory	C-002, C-003	Agent/Output을 실제 드롭·연결하고 mock run 진행 및 지시 접수 표시
C-005	P1	Zod 정의·명령·이벤트 schema 및 codegen	C-001	JSON Schema와 TS type의 fixture parity, unknown field 거절
C-006	P1	4종 노드·포트·capability 링크	C-004, C-005	control과 delegate/question을 구별하고 연결 허용을 드러냄
C-007	P1	Loop 영역·본문 membership 편집	C-006	그룹 이동과 loopId 편집을 구별하고 경계 위반 표시
C-008	P1	정적 검증기·IR compiler	C-005, C-007	유효 정의는 IR, 잘못된 cycle/port/ref는 위치 포함 오류
C-009	P1	저장/복제/import/export·정의/layout 분리	C-005, C-006	좌표 변경은 실행 hash 불변, ID 복제 충돌 없음
C-010	P1	Undo/Redo·키보드 연결·폼·IME	C-006, C-009	한 번 드래그를 한 undo로 처리, IME Enter 오동작 없음
C-011	P2	SQLite migrations·Repository·트랜잭션	C-005	event/projection/artifact 원자 commit과 rollback 테스트
C-012	P2	Coordinator decide/reduce·명령 dedupe	C-011	동일 command 재전송과 충돌/취소 fence의 순서가 결정적
C-013	P2	EffectDispatcher·epoch·activation key	C-012	commit 전 effect 없음, duplicate activation 중복 Task 없음
C-014	P2	LangGraph GraphRunner 제한 IR 실행	C-008, C-013	GraphRunner 결과가 Coordinator 경로를 거쳐야 완료됨
C-015	P2	REST receipt·snapshot·SSE watermark	C-012	replay/live 전환 gap 없이 중복 제거, 202를 완료로 오인 안 함
C-016	P2	실행 트리·노드 집계·WaitingReason	C-015	같은 node의 두 execution과 대기 링크를 별도 표시
C-017	P3	Pi worker 생명주기와 IPC schema	C-002, C-013	unexpected exit/invalid IPC/worker binding 검증
C-018	P3	request permit·context snapshot·stream bridge	C-017, C-012	request 전에 확정된 snapshot을 실제 provider 호출에 사용
C-019	P3	도구 훅 mapping·실행 allowlist	C-017	Pi start 이후 block돼도 실제 tool.started 없음

ID	단계	작업	의존	완료 조건
C-020	P3	모델 2프로필·usage normalization·live smoke	C-018, C-019	지원 기능·모델ID·usage·abort를 각 provider로 검사
C-021	P4	Task 담당·시도 소유권·취소 결과	C-012	한 Task에 활성 시도 2개 불가, 자동 재할당 없음
C-022	P4	spawn/await·부모 자식 registry	C-021, C-017	두 child 병렬, parent wait는 모델 permit 미점유
C-023	P4	질문·응답 Task·reply resolver	C-021, C-022	끝난 역할 새 responder, 답변/timeout/cancel 단회 종결
C-024	P4	산출물 버전·제출·read 추적	C-011, C-021	동시 helper 출력 이름 충돌 없음, 미제출 버전의 control 전달 없음
C-025	P4	협업 도구와 Task Board/메시지 UI	C-022, C-023, C-024	질문에서 responder와 근거 버전으로 이동 가능
C-026	P5	입력 바인딩·회차 pinning	C-024, C-014	reviewer가 이전 draft가 아닌 이번 제출 draft를 읽음
C-027	P5	review schema·기준 버전·stale 판정	C-026	잘못된 target/criteria/JSON은 pass로 승격 안 됨
C-028	P5	Loop cap·다음 회차·수정 작업	C-027	cap에서 approval, 승인만으로 한도 우회 불가
C-029	P5	Approval snapshot과 최종 Output	C-027, C-028	동일 승인 2회 실행 없음, stale 대상 거절
C-030	P5	Artifact diff·lineage 기반 상세 화면	C-024, C-027	draft@1→review@1→draft@2 경로를 확인
C-031	P6	Steer enqueue·delivery receipt	C-018, C-025	실행 중 접수되고 다음 request에 정확한 directive IDs 포함
C-032	P6	Pause/drain/resume 경계	C-022, C-031	대기 child 때문에 pause 교착 없음, paused 후 신규 effects 없음
C-033	P6	전체·개별 cancel과 후손 정리	C-021, C-022, C-023	후손/도구/질문 resolver 종료 후 취소 완료
C-034	P6	명시적 재시작과 이전 epoch 차단	C-033	부모가 대기하는 Task만 새 시도, 이전 결과는 감사만
C-035	P6	미래 config patch·범위 미리보기	C-031	active config 불변, future node activation에만 반영
C-036	P6	지시/기준 변경과 산출물 연결	C-027, C-030, C-031	접수 시각만으로 이전 요청 결과를 지시에 연결하지 않음
C-037	P7	예산 예약·정산·설정 변경	C-020, C-035	동시 요청 초과 admission 방지, unknown usage의 거짓 환급 없음
C-038	P7	wait graph·총량·deadline·no progress	C-023, C-028	대기 cycle과 새 responder pingpong 모두 유한 상한

ID	단계	작업	의존	완료 조건
C-039	P7	경합 property test·late results	C-031, C-032, C-033, C-034, C-037	seed 재현 가능한 명령 interleaving에 불변식 유지
C-040	P7	crash interrupted·기록 export/reopen	C-011, C-033	재시작 후 pending effect 실행 없음, read-only 기록 복원
C-041	P7	backpressure·재접속·마지막 텍스트 복구	C-015, C-016	slow client 메모리 제한, 누락 domain event 재조회
C-042	P8	local session·Origin/Host/CSRF·키 격리	C-015, C-017	임의 origin/host와 query secret 누출 검사
C-043	P8	문서 import·안전 URL reader	C-019, C-024, C-042	IPv4/IPv6/redirect/DNS pinning/size/MIME/HTML 실행 차단
C-044	P8	SearchProvider mock+Brave live	C-043	키 없는 corpus search와 명시적 외부 검색 선택 구분
C-045	P8	CLI 동일 command/query 경로	C-015, C-040	validate/dev/run/inspect/export, 승인 대기와 SIGINT 계약
C-046	P9	public DemoDriver와 fixture pack	C-004, C-031, C-029	비밀키/로컬 서버 없이 D&D와 Reins가 모의 결과를 바꿈
C-047	P9	Storybook 상태·Playwright 키보드/IME	C-010, C-016, C-025, C-031	포인터 없이 핵심 흐름 완주, 단순 axe 검사 외 수동 검증
C-048	P9	부하·가상화·profiling 예산	C-041, C-047	고정 fixture/장비/build에서 수치와 flamegraph 기록
C-049	P9	통합 release gate·CI·SBOM	C-039, C-040, C-042, C-046, C-047, C-048	모든 요구 테스트 연결, 실제 LLM smoke는 별도 결과
C-050	P9	포트폴리오 사례 문서·known limitations	C-049	성능/기능 주장을 증거와 연결, 미구현 v0.2 기능 혼동 없음

7. 개발 분담과 통합 규칙

1인 개발이더라도 역할을 분리한다. Bell은 제품·용어·UX·수용 기준의 최종 결정자다. 구현 에이전트는 합의된 spec와 허용 파일 범위 안에서 구현한다. 별도 테스트/리뷰 역할은 happy path뿐 아니라 경합·거절·오용을 검사한다.

동시에 작업하기 좋은 경계는 Canvas/UI, Coordinator/Storage, Pi Adapter/Testkit이다. `contracts`와 공통 상태 전이는 동시 수정하지 않고 하나의 spec change가 먼저 merge되어야 한다. feature branch마다 contractVersion과 base revision을 기록한다.

PR에는 requirement IDs, 실패 테스트 설명, 실행한 명령과 결과, 화면 변경 시 story/trace, 위험·롤백을 적는다. 실제 실행하지 않은 테스트를 통과로 적지 않는다. 비용이 발생하는 live smoke는 별도 opt-in이며 API key를 로그·PR에 올리지 않는다.

8. CI 파이프라인

아래 스크립트 이름은 구현 목표다. 최초 저장소에서 실제 명령으로 정의하고 CI에서 검증해야 한다.

```
pnpm install --frozen-lockfile
pnpm spec:check            # 요구·태스크·테스트 ID와 명세 링크
pnpm contracts:check       # codegen diff + fixtures + schema parity
pnpm lint                 # Biome
pnpm typecheck            # strict + 패키지 경계
pnpm test:unit
pnpm test:contract
pnpm test:integration
pnpm build
pnpm test:e2e
pnpm test:a11y
pnpm test:performance      # nightly/release의 고정 부하
pnpm test:live             # 수동/예산 제한, 일반 PR에서 자동 호출 금지
```

PR gate는 unit/contract/integration/build와 핵심 E2E다. release gate에는 a11y·performance·라이선스·SBOM·live smoke 결과를 더한다. real LLM 결과가 flaky라고 deterministic 취소/버전 테스트를 retry로 숨기지 않는다.

외부 PR은 production key를 받지 않는다. GitHub Actions permission은 필요한 read/write만 부여하고, 공개 action은 검토한 commit SHA로 고정한다. dependency update는 그룹 PR로 만들고 contract suite를 통과해야 반영한다. provenance/보안 스캔 미확인 결과는 release note에 남긴다.

9. 리스크와 중단 기준

리스크	조기 신호	대응	release 차단 여부
Pi hook 의미/버전 불일치	blocked tool에도 started, 지시 전달 시점 불명확	pinned source contract 재검증, adapter 수정	차단
상태 중복 소유	DB와 UI/Pi의 completed가 어긋남	Coordinator 밖 mutation 금지·reducer 테스트	차단
부모 대기 슬롯 고갈	자식은 queued, 부모 전부 waiting	모델 permit과 worker residency 분리	차단
late callback 부활	cancelled task가 completed로 변함	epoch fence·idempotency·race property test	차단
비용 과소 집계	unknown/cached usage가 0 또는 중복	raw+normalized, unknown budget 보수 처리	hard cap 주장 시 차단
그래프 UI와 실행 불일치	연결 바꿔도 같은 순서	compiler-E2E fixture 연결	차단
스티어링의 거짓 적용	이전 request 결과에 새 지시 인과	ContextSnapshot 기반 연결	차단
로컬 키 노출/SSRF	export/client/로그에 key, 사설 주소 fetch	security suite·endpoint 정책	차단

리스크	조기 신호	대응	release 차단 여부
유료 기능 종속	Pro 예제/외부 서비스 없이 demo 불가	공개 API 기반 구현·mock 대체	차단
UI 범위 팽창	IDE/문서 편집/멀티플레이 추가	Change Request로 후속 마일스톤 이동	scope review

10. 출시 판정과 포트폴리오 산출물

출시 보고서는 기능 체크 표시만이 아니라 AC별 테스트/trace/스크린샷/fixture hash를 연결한다. 공개 사례 문서에는 문제 정의, 상태 소유권 결정, D&D와 실행의 경계, 스티어링 적용 예, 접근성·성능 측정 환경, 실패했던 설계와 수정 이유를 기록한다.

README에서 실행 가능한 기능과 계획된 기능을 다른 표로 구분한다. `Local-first`는 모델 추론도 전부 로컬이라는 뜻이 아니며, 지원 provider와 외부 전송 범위를 명시한다. 이미 수행한 외부 작업을 되돌릴 수 있다는 오해를 만드는 replay 표현을 피한다.

11. MVP 다음 구현: v0.2 Decision switching

MVP 완료 후 첫 후속 기능으로 **Local/Jev 수동 switching**을 진행한다. DecisionQuestion, 후보 집합, response schema, capability registry, timeout, privacy policy를 먼저 명세한다. 해당 로컬 모델이 지원하지 않는 질문을 형식만 맞춰 전달하지 않는다.

1차 수용 기준은 같은 질문·후보에 두 adapter를 연결하고, 결과·지연·오류·근거 수준을 비교 가능한 형태로 기록하는 것이다. 자동 라우팅의 비용 절감이나 Hybrid threshold 품질은 별도 평가 단계다. LiteLLM 도입은 이 기능의 필수 조건이 아니다.

PART 05

테스트 전략

불변식 · 경합 · Mock과 실제 LLM 평가 · 성능 · 보안

1. 무엇을 테스트하는가

제품 기능은 아직 구현되지 않았다. 이 문서 패키지에 있는 test-catalog는 **구현해야 할 테스트 설계**다. 앱 테스트를 실행했다거나 통과했다는 보고가 아니다. 패키지 생성 시 수행하는 JSON·참조·ID 검사는 문서 일관성 검증이며 별도 Validation Report에 기록한다.

단위 테스트는 순수 decide/reduce, graph parser, budget allocator, version binding, scope matcher를 검증한다. contract 테스트는 JSON schema, Pi source event mapping, provider usage normalization, worker IPC, API receipt를 검증한다. 통합은 SQLite+Coordinator+mock effects, E2E는 브라우저에서 D&D와 Reins의 전체 흐름, live는 실제 provider 기능, performance는 고정된 부하 fixture를 사용한다.

2. 도구와 역할

도구	역할	주의
Vitest	단위·계약·통합, fake timers, coverage	timer advance 전에 microtask/외부 completion barrier를 명시
React Testing Library	접근 가능한 이름·사용자 행동 기반 DOM 검사	내부 store 구현에만 결합하지 않음
Playwright	Canvas 좌표·포인터·키보드·브라우저 streaming	jsdom만으로 실제 SVG/viewport 행동을 증명하지 않음
MSW	API 응답/오류·stream 상태 fixture	Coordinator 자체 논리는 mocked HTTP가 아닌 실제 통합으로 별도 검증
fast-check	명령 sequence와 경합 불변식·seed 재현	실제 LLM의 품질을 property로 판정하지 않음
Storybook	상태별 UI 계약·개입 패널·긴 문구	유료 서비스 없이 로컬 build+브라우저 테스트
axe / Playwright	자동 접근성 위반 탐지	keyboard/IME/읽기 순서 수동 점검도 필수

3. 제어 가능한 Testkit

`FakeClock`은 wall/monotonic 시간을 각각 제공한다. `SequentialIdFactory`는 알려진 ID를 반환한다. `ScriptedAgentAdapter`는 요청 준비/응답 시작/도구 제안/답변/종료를 테스트가 결정하게 한다. `DeferredBarrier`로 "이 시점에 cancel이 들어옴"을 정확히 재현한다.

MockAdapter는 같은 roleId에 고정 문자열을 출력하는 것만으로 끝내지 않는다. 요청의 directiveIds, 실제 input version, graph 순서에 반응해야 D&D·스티어링 테스트가 의미 있다. Public DemoDriver는 같은 의미 계약을 사용하되 별도 worker/타이머로 시각적인 진행 시간을 제공한다.

네트워크 금지 테스트는 예상하지 않은 fetch를 실패로 처리한다. live suite만 명시적 key/env로 활성화한다. test log의 예산·usage는 fixture임을 표시한다.

4. 핵심 불변식

ID	불변식
INV-01	같은 Task에 active attempt는 최대 하나
INV-02	취소 fence 뒤 대상 scope에서 신규 effect admission 없음
INV-03	parent 완료/취소 시 required owned children은 모두 정리됨
INV-04	동일 commandId+payload는 한 receipt/효과, 다른 payload는 충돌
INV-05	artifact version 본문/hash/원천은 불변
INV-06	review의 target와 criteria는 실제 검토한 snapshot과 일치
INV-07	directive 직접 적용은 해당 request snapshot 포함에서만 발생
INV-08	budget spent/reserved/unknown은 음수가 아니고 중복 정산되지 않음
INV-09	모델 permit 수≤3, 기다리는 parent는 모델 permit을 점유하지 않음
INV-10	message-triggered spawn도 depth/total 제한을 사용
INV-11	terminal Run/Execution은 늦은 결과로 running/completed에 부활하지 않음
INV-12	layout 변경은 definitionHash를 바꾸지 않음
INV-13	rollback된 트랜잭션에서 provider/tool 효과가 발생하지 않음
INV-14	replay한 projection은 최종 DB projection과 동일
INV-15	provider raw usage 미확인을 0 비용으로 환산하지 않음
INV-16	관측되지 않은 모델 내부 생각·행동을 UI가 생성해 표시하지 않음

5. 테스트 피라미드가 아니라 경계별 증거

모든 테스트를 E2E로 작성하지 않는다. `TC-RNS-001-N`은 Coordinator의 지시 큐/요청 snapshot을 통합으로 검증하고, 그 결과 receipt가 UI에 어떻게 보이는지는 별도의 Reins E2E로 검증한다. 화면 screenshot 하나가 예산 원자성을 보장하지 않으며, reducer test 하나가 실제 keyboard 조작을 증명하지 않는다.

Coverage 목표는 core/contracts line90/branch85로 시작한다. critical invariant에 음성·경합 테스트가 없다면 coverage를 넘겨도 release를 차단한다. PR에서는 안정된 mock suite, nightly/release에서는 seeded model-based와 성능 시험, live provider는 opt-in으로 실행한다.

6. 스티어링의 Given/When/Then

```
@CRL-RNS-001 @CRL-RNS-010
Scenario: 지시를 받은 요청과 시간상 뒤의 요청을 구분한다
  Given 모델 요청 A의 snapshot이 지시 없이 이미 확정되었다
  When 사용자가 지시 D를 접수한다
  And 요청 A가 초안 V1을 생성한다
  Then V1에는 D의 직접 적용 연결이 없어야 한다
  When 다음 요청 B가 D를 포함한 snapshot으로 시작한다
  And B가 초안 V2를 생성한다
  Then D는 B에 전달된 상태여야 한다
  And V2는 B와 D의 직접 적용 경로를 보여야 한다
  And 지시 준수 판정은 별도 평가 상태여야 한다
```

같은 형식으로 partial cancel 후 parent unblock, budget patch vs admission, approval stale, SSE replay gap을 작성한다. .feature 파일은 수용 기준의 표현이며 이 패키지가 Cucumber 실행기를 구현했다는 뜻은 아니다. 실제 suite는 동일 ID를 가진 Vitest/Playwright 테스트로 연결한다.

7. 실제 LLM 평가

두 모델 프로필의 tool schema, usage, context injection, abort는 API contract smoke다. 작업 산출물 품질은 별도 `eval`이다. 고정된 입력 자료와 같은 역할·기준을 사용하고, 결과를 model ID/config/snapshot과 함께 보관한다.

초기 비교는 단일 agent / 고정 팀 / 스티어링한 팀으로 한다. 평가 항목은 근거 적합성, 완료 기준 충족, 수정 횟수, 사용자 개입 횟수, 총 시간, 알려진 usage와 비용이다. 한두 실행의 결과를 통계적 우월성으로 일반화하지 않는다. 토큰이 줄어도 품질이 나빠졌다면 비용 절감 성공으로 표시하지 않는다.

8. UX·성능 검증

Playwright는 1280/1440/1920 폭, keyboard-only flow, drag/connect/delete/undo, IME composition, dialog focus return, reduced motion을 검사한다. 실제 브라우저로 Chromium과 Firefox를 필수, WebKit은 지원 선언 전 smoke한다. Storybook은 waiting reason/긴 한국어·일본어/네트워크 단절/expired directive/unknown usage 상태를 고정한다.

성능 fixture는 20 nodes, 100 executions, 10k events, 50 incoming frames/s다. production build에서 테스트하고 브라우저/CPU/메모리/실행시간을 기록한다. 로컬 command receipt p95 200ms, event-to-UI p95 300ms를 초기 목표로 둔다. 스트림 텍스트 수신량이 가변이면 함께 기록한다. 실제 모델 지연을 frontend 지연으로 섞지 않는다.

9. 증거 구조

각 feature의 evidence에는 테스트 명령, exit code, 실행 일시, fixture/schema commit hash, 실패 시 trace, live 여부를 둔다. 비밀키나 private transcript를 공개하지 않는다. RED 증거는 임시 feature branch/작업 기록에 보관하고 보호된 main을 실패 상태로 유지할 필요는 없다.

`test-catalog.json`의 status는 구현 테스트를 실행하기 전까지 planned다. evidence가 없는 테스트를 passed로 변경하지 않는다. `traceability.json`은 요구사항의 완성도를 수치로 과장하기 위한 것이 아니라 빠진 계약을 찾는 자료다.

PART 06

아키텍처 결정 기록

13개 ADR · 선택 근거 · 대안 · 제약

상태는 모두 **설계 채택안 / 구현 검증 전**이다. 사용자 합의와 새 기술 선택의 근거를 남긴다. 변경은 기존 기록 삭제가 아니라 새 ADR로 대체한다.

ADR-001: 브랜드 은유는 경계와 조작에만 적용

결정: Corral은 root 팀·그래프·공유 한도, Loop는 반복, Studio는 UI, Reins는 제어 패널로 정한다. Task Board·Artifacts·Run은 일반 용어를 유지한다.

이유: 팀과 loop를 동일 타입으로 부르거나 모든 명칭을 말/마장 은유로 치환하면 API와 UI가 모호해진다.

결과·제약: 중첩 Corral·Hitching Post·Trough를 MVP entity로 추가하지 않는다.

ADR-002: D&D와 Reins는 MVP 필수

결정: P0부터 캔버스→검증→mock 실행의 수직 슬라이스를 만든다. 완성된 backend 뒤에 UI를 붙이는 순서로 미루지 않는다.

이유: 사용자 목표가 기능적 도구와 프런트엔드 포트폴리오를 함께 포함한다.

결과·제약: 범위 축소는 provider·node 범용성·외부 연동을 대상으로 한다.

ADR-003: 하나의 Coordinator가 도메인 상태를 확정

결정: 명령을 직렬화하고 events+projection+artifact+effect intent를 SQLite txn에서 commit한다.

이유: Pi·graph state·UI 상태가 각각 Task 완료를 정하면 취소/patch 경합을 설명할 수 없다.

결과·제약: 역할 분리는 유지하되 authoritative writers를 늘리지 않는다.

ADR-004: LangGraph는 정적 flow runner로 제한

결정: v1.1의 LangGraph 채택은 유지하되 도메인 상태·durable checkpointer·interrupt 기반 스티어링은 쓰지 않는다.

이유: 그래프 순회 라이브러리와 업무 제어를 혼동하지 않는다. crash resume은 범위 밖이다.

결과·제약: P0/P1에서 node activation 멱등성과 외부 gate를 증명하지 못하면 ADR를 갱신하고 제한형 runner 대안을 평가한다. 몰래 이중 scheduler를 만들지 않는다.

ADR-005: Pi agent-core를 worker로 감싼다

결정: coding-agent 전체 대신 agent-core+pi-ai, 명시한 도구만 제공한다. worker는 host RPC로 신규 효과를 요청한다.

이유: 원하는 역할은 코딩 전용이 아니며, 권한과 사용량을 host가 검증해야 한다.

결과·제약: child process는 security sandbox가 아니다. untrusted plugin·shell 없음. worker admission overhead는 측정한다.

ADR-006: 지시와 설정 변경을 다른 타입으로 관리

결정: directive는 특정 execution의 다음 요청, patch는 node의 미래 activation 또는 run limit에 적용한다.

이유: 모델/프롬프트를 현재 요청 도중 바꾸는 것은 안전 경계와 모델 컨텍스트를 흐린다.

결과·제약: 템플릿 원본 불변, run-only overlay. 도구 권한 확대는 MVP에서 금지.

ADR-007: 일시정지·취소·재시작·crash resume을 구별

결정: pause는 새 effect를 막고 경계 대기, cancel은 termination, restart는 허용된 미완료 task의 새 attempt다. crash는 interrupted 기록 열람만 허용.

이유: 모든 조작을 stop/replay 하나로 합치면 외부 효과를 중복 수행할 수 있다.

결과·제약: 취소됐다고 모델 공급자 내부 과금 중지까지 주장하지 않는다.

ADR-008: 공유 예산에는 예약·정산을 사용

결정: 모델 dispatch 전 한도 검사와 reservation, 완료 후 settlement. unknown usage는 별도 상태로 유지.

이유: 병렬 호출이 동일 잔여량을 보고 모두 시작하면 사후 집계만으로 상한을 지킬 수 없다.

결과·제약: 외부 공급자 미보고/불완전 취소의 비용까지 엄격히 보장하지 않고 새 admission을 안전하게 보류한다.

ADR-009: 질문은 추적되는 응답 작업이다

결정: 질문 수신 역할에 새 response task/instance를 만들고 scope 입력만 준다. finished 실행을 다시 살아있게 만들지 않는다.

이유: 현재/종료된 receiver의 mailbox 처리와 재개 정책을 통일하는 부담을 줄인다.

결과·제약: 새 응답도 spawn·depth·budget·round-trip 상한에 들어간다. 비용 최적화 상주 에이전트는 후속.

ADR-010: 산출물은 불변 버전과 명시적 제출

결정: 본문은 SQLite에 넣고 write/submit/review/approval을 구별한다. 현재 반복 선행 출력과 이전 반복 입력 바인딩을 분리한다.

이유: latest 전역 조회는 reviewer가 다른 버전을 보는 버그를 만든다. 파일과 DB의 이중 commit도 피한다.

결과·제약: 큰 바이너리 산출물과 외부 blob 저장은 후속.

ADR-011: 관측과 실제 적용을 구분

결정: domain SSE에는 commit된 event만, token frame은 transient. directive lineage는 model context snapshot으로 만든다.

이유: 시간상 나중이라는 사실만으로 지시가 원인이라고 할 수 없다. Pi tool start도 허용 전 관측일 수 있다.

결과·제약: provenance와 지시 준수 판정은 다른 필드로 관리한다.

ADR-012: 공개 데모와 실제 실행을 명확히 분리

결정: 동일 Studio에 mock RuntimeTransport를 연결해 static demo를 제공한다. live는 로컬 auth 서버에서만 API 키 사용.

이유: 방문자가 키 없이 프런트엔드 역량과 동작을 체험할 수 있다. 비밀키 유출/무제한 비용을 피한다.

결과·제약: mock 표시 상시 노출. live 품질 검증의 대체로 사용하지 않는다.

ADR-013: SDD와 TDD에 추적성과 반례를 요구

결정: REQ→scenario→contract→test→task의 mapping을 유지한다. 테스트를 실제 실패시킨 증거와 반례를 PR에 남긴다.

이유: 에이전트가 구현과 테스트를 동시에 작성하면 잘못된 가정을 양쪽에 복제할 수 있다.

결과·제약: 코드 리뷰는 spec 준수와 spec 자체 타당성을 별도로 확인한다.

PART 07

라이브러리·출처

OSS 선정 · 유지보수 관측 · 버전 잠금 · 원자료

> 기준일 2026-10-14. **선정안이며 설치된 lockfile·호환성 검사 결과가 아니다.** 공식 문서와 일부 릴리스 API를 직접 확인했다. 조회한 저장소의 최근 릴리스와 npm 패키지의 최신 버전을 혼동하지 않는다. 실시간 정보는 착수 시 다시 확인한다.

1. 선정 정책

핵심 의존성은 오픈소스 라이선스, 공개 소스·문서·변경 이력, 교체 가능한 경계, TypeScript·로컬 실행 적합성을 기준으로 선정했다. star 수를 품질 지표로 사용하지 않는다. 빈번한 릴리스만으로 안정성도 보장되지 않는다.

릴리스 날짜까지 확인한 핵심군: Pi, xyflow, LangGraph.js 저장소, Fastify, Vitest. 나머지는 공식 문서·저장소를 확인한 후보이며 모두 같은 수준으로 최신 활동을 검증했다고 주장하지 않는다. P0의 버전 잠금 게이트에서 각 패키지의 최신 안정 릴리스·최근 변경·보안 공지·라이선스를 보충한다.

2. 도입할 라이브러리

계층	채택	역할·선택 이유	라이선스/주의	근거
실행 환경	Node.js 24 LTS, pnpm workspace	브라우저/서버/worker의 TS 모노레포	배포판·전이 의존성 라이선스 별도 확인	S26,S29
UI	React 19.2 계열, TypeScript, Vite 8.1 계열	SEO보다 상호작용 중심 SPA; 서버 수명과 UI build 분리	React/Vite MIT, TS Apache-2.0; 정확한 patch 잠금	S27,S28
캔버스	@xyflow/react	노드·포트·연결·선택·뷰포트. 도메인 그래프는 자체 정의	MIT OSS core. Pro 예제 코드 구매를 전제로 하지 않음	S01~03,S35
UI 상태	Zustand	editor / live projection / Reins / stream store 분리	MIT; React Flow 내부 상태와 중복 mutable truth 금지	S08
조회 캐시	@tanstack/react-query	정의 목록, 과거 Run, 산출물 요청	MIT; live run projection은 별도 store	S09
가상화	@tanstack/react-virtual	긴 타임라인·트리 행	MIT; 현재 focus/선택 행이 사라지는 경우 처리	S10
UI primitives	Radix Primitives + Tailwind CSS	접근 가능한 Dialog, Tabs, Tooltip; Corral만의 컴포넌트 디자인	MIT 계열; 전체 템플릿 복사보다 필요한 primitive만	S11,S30

계층	채택	역할·선택 이유	라이선스/주의	근거
폼	react-hook-form	역할·모델·한도·Reins 폼의 dirty/validation 상태	MIT; 서버 validation은 별도 필수	S12
계약	Zod 4	REST/IPC/정의/command validation 및 JSON Schema 생성	MIT; TypeBox는 Pi 어댑터 내부 경계에 제한	S13,S07
로컬 API	Fastify 5	세션, 명령, 조회, SSE, 일관된 오류	MIT; SSE 취소/정적 UI/보안 plugin 호환성 검사	S14,S38
그래프 실행	@langchain/langgraph	검증된 정적 그래프/제한된 loop 실행	MIT. 제품의 authoritative task/patch 상태를 중복 소유하지 않음	S04,S05,S40
에이전트	@earendil-works/pi-agent-core + pi-ai	모델 tool loop·stream·훅·취소	MIT, 확인 릴리스1.2.0; coding-agent의 임의 shell loader는 제외	S06,S07,S36
저장	SQLite + better-sqlite3 + Drizzle	로컬 영속 이벤트·투영·텍스트 산출물의 atomic commit	SQLite public domain / driver MIT / ORM Apache-2.0	S15,S20~23,S34
단위·통합	Vitest	pure reducer, timeout·fake timer, DB/API 통합	MIT; 관측 릴리스5.0.4를 무조건 채택하지 말고 호환 검증	S16,S39
성질 테스트	fast-check	무작위 command interleaving·예산·취소 불변식	MIT; seed와 shrink 결과 보존	S17
브라우저	Playwright + axe 통합	실제 D&D·IME·재접속·접근성·취소 경합	Playwright Apache-2.0. axe 전이 라이선스 P0 기록	S18
HTTP mock	MSW	Studio 단독 개발, 실패·지연·충돌 응답	MIT; 런타임 테스트의 Agent mock과 역할 구분	S19
컴포넌트	Storybook + Testing Library	상태별 Reins·노드·대기행 테스트	각 패키지 라이선스 P0 잠금. 유료 hosted CI 불필요	S31
코드 품질	Biome, TypeScript strict	일관된 formatting, lint, type check	문서/JSON/Mermaid 검사도 별도 스크립트	S32

`react-markdown + remark-gfm`, 텍스트 diff 라이브러리, CLI 인자 파서는 작은 보조 의존성으로만 추가한다. P0에서 라이선스·최근 유지보수 확인 후 정확한 버전을 결정한다. HTML passthrough·실행 가능한 Markdown 확장은 사용하지 않는다. 필요 없는 편집기 SDK·거대한 아이콘 패키지는 초기 필수 의존성으로 두지 않는다.

2.1 두 종류의 schema 도구를 무분별하게 섞지 않는다

Corral 도메인 스키마의 원본은 Zod다. Pi가 요구하는 TypeBox 도구 parameter는 어댑터 안에서 생성하거나 얇게 매핑한다. 양쪽을 별개 수작업 사양으로 유지하지 않는다. 변환 가능한 제한된 JSON Schema subset을 정하고 golden round-trip test로 동등성을 확인한다. 이 팩의 JSON Schema는 그 전 단계의 **계약 초안**이며, 구현 착수 후 Zod 원본에서 생성하도록 이동한다.

3. 유지보수 증거와 버전 잠금

프로젝트	조회에서 관측한 것	이 사실이 보장하지 않는 것
Pi	v1.2.0, 2026-10-12. 태그의 agent-core package는 MIT·Node >=22.19.0	Corral worker/steering 어댑터의 완성, 모든 provider 취소 품질
xyflow	@xyflow/system@0.0.83, 2026-09-24	@xyflow/react의 현재 버전이 0.0.83이라는 뜻이 아님
LangGraph.js	@langchain/vue@1.2.4, 2026-10-12	@langchain/langgraph 런타임이 같은 버전이라는 뜻이 아님
Fastify	v5.12.7, 2026-10-13	모든 SSE/static/session plugin과의 자동 호환
Vitest	v5.0.4, 2026-10-12	Pi repo의 개발용 Vitest와 동일할 필요 없음; Vite/Node 호환 검증 필요
React / Vite	공식 페이지 React19.2, Vite8.1 발표 확인	이후 patch의 최신성이나 무결함 보장 아님

정확한 설치 버전은 P0에서 `package.json + pnpm-lock.yaml + runtime version file + dependency-evidence.json`으로 고정한다. 버전 evidence에는 license, integrity, upstream tag/commit, release date, 확인일, peer/Node 요구, 검사한 테스트를 적는다. `latest`, 무제한 *, RC 문서 예시를 그대로 설치하지 않는다.

특히 Drizzle 공식 SQLite 문서에 `@rc` 설치 예시가 있으므로, 최신 웹 문서를 읽었다는 이유로 pre-release를 제품 기본값으로 채택하지 않는다. 안정판 태그의 migration/API를 검증한다. 드라이버는 Node24/Linux/Windows WSL 환경에서 설치·transaction·shutdown 테스트한다.

4. 넣지 않는 것과 이유

의존성/기능	MVP 처리	재검토 조건
Next.js	사용하지 않음	공개 콘텐츠 SEO나 서버 렌더링이 실제 요구일 때. 로컬 장수 Run의 수명은 Next 요청 수명에 묶지 않음
Tauri / Electron	사용하지 않음	로컬 웹 MVP 후 설치·OS 통합이 필요할 때
XState / Redux / RxJS	추가하지 않음	이미 정의한 Coordinator·Zustand를 대체할 구체적 이익이 있을 때
Redis / BullMQ / Kafka / Temporal	추가하지 않음	분산 실행·durable resume·멀티유저가 실제 요구일 때

의존성/기능	MVP 처리	재검토 조건
LiteLLM	후속	provider/유저 비용 운영 필요가 커졌을 때
Local / Jev	v0.2 명시 마일스톤	협업·Reins MVP 완료 후 동일 질문 계약을 갖춘 수동 전환부터
Langfuse	자체 Run 기록을 대체하지 않음	외부 observability export 필요 시
Spec Kit / OpenSpec	선택 문서 도우미	저장소 명세·테스트 규칙을 훼손하지 않는 경우. 필수 런타임 없음
React Flow Pro / Chromatic	필수 아님	유료 예제·서비스의 가치를 별도로 판단할 때

5. 오픈소스와 외부 서비스의 경계

Corral 소스의 배포 라이선스는 **MIT를 제안**한다. 아직 실제 저장소의 소유자·저작권 고지·배포 절차를 확정했다는 뜻은 아니다. npm scope `@corral/*`, CLI bin, 상표·도메인 사용 가능성도 이 조사에서 검증하지 않았다. 우선 private workspace로 사용한다.

자체 실행·UI·저장·테스트 경로는 OSS 구성으로 만든다. 실제 LLM 호출과 Brave Search는 별도의 외부 서비스이며 키·이용 조건·과금이 적용될 수 있다. mock/corpus demo는 그 비용 없이 작동해야 한다. 서비스 구독의 비공식 OAuth 재사용을 기본 인증으로 설계하지 않는다.

배포 전 직접·전이 의존성의 SPDX 목록, 원문 license/NOTICE, 공급망 audit를 만든다. 보조 패키지의 라이선스는 여기서 미검증인 경우 그 사실을 유지한다. 법률상 사용 가능성 전체를 이 표가 보장하지 않는다.

6. 출처와 증거 원장

아래 출처는 기능 사실 확인용이다. 제품의 새 상태 모델·예산 정책·작업 분해·일정은 Corral 설계 판단이다. 사용자 PDF는 원문을 이 ZIP에 중복 포함하지 않고 문서명·절을 참조한다.

[B1] 합의 기준: agent-harness-mvp-spec_v1.1.pdf

사용자 첨부, 2026-10-14, 14쪽

확인일: 2026-10-14. 원문 1부 §1~8 기준. 이 문서 팩의 새 결정은 명시적으로 구분한다.

[B2] 이전 리뷰와 역제안

사용자 첨부 plan-review.md / mvp-spec.md / v1.0 PDF

확인일: 2026-10-14. D&D 연기·YAML-only 부분은 최신 합의로 대체됨. 상태·테스트 분석만 참고.

[S01] React Flow 공식 문서

`https://reactflow.dev/learn`

확인일: 2026-10-14. D&D 캔버스 기반. 그래프 실행기는 아님.

[S02] React Flow 성능

`https://reactflow.dev/learn/advanced-use/performance`

확인일: 2026-10-14. 컴포넌트 메모이제이션과 좁은 상태 구독.

[S03] React Flow 접근성

`https://reactflow.dev/learn/advanced-use/accessibility`

확인일: 2026-10-14. 키보드·스크린리더 구성.

[S04] LangGraph 영속성

`https://docs.langchain.com/oss/javascript/langgraph/persistence`

확인일: 2026-10-14. checkpointer와 store의 다른 책임.

[S05] LangGraph interrupts

`https://docs.langchain.com/oss/javascript/langgraph/interrupts`

확인일: 2026-10-14. resume 시 노드 재진입과 부작용 주의.

[S06] Pi agent-core, 확인한 릴리스 v1.2.0

`https://github.com/earendil-works/pi/blob/v1.2.0/packages/agent/README.md`

확인일: 2026-10-14. 훅·요청 준비·도구 이벤트·취소. main 문서도 함께 확인, 태그 차이는 P0 검사.

[S07] Pi package metadata v1.2.0

`https://github.com/earendil-works/pi/blob/v1.2.0/packages/agent/package.json`

확인일: 2026-10-14. MIT, Node >=22.19.0, pi-ai와 typebox 의존.

[S08] Zustand 공식 저장소

`https://github.com/pmndrs/zustand`

확인일: 2026-10-14. MIT; vanilla store와 selector 기반 상태.

[S09] TanStack Query

`https://tanstack.com/query/latest/docs/framework/react/overview`

확인일: 2026-10-14. 서버 조회 상태. 라이선스: https://github.com/TanStack/query

[S10] TanStack Virtual

`https://tanstack.com/virtual/latest`

확인일: 2026-10-14. 긴 실행 트리·타임라인 가상화. 라이선스: https://github.com/TanStack/virtual

[S11] Radix Primitives

`https://www.radix-ui.com/primitives/docs/overview/introduction`

확인일: 2026-10-14. Dialog·Tabs 등 저수준 UI. MIT: https://github.com/radix-ui/primitives

[S12] React Hook Form

https://github.com/react-hook-form/react-hook-form

확인일: 2026-10-14. 폼 상태·검증, MIT.

[S13] Zod

https://zod.dev/

확인일: 2026-10-14. v4 stable, JSON Schema 지원. MIT: https://github.com/colinhacks/zod

[S14] Fastify

https://fastify.dev/

확인일: 2026-10-14. 로컬 Node API 기반. MIT: https://github.com/fastify/fastify

[S15] Drizzle SQLite

https://orm.drizzle.team/docs/sqlite/get-started-sqlite

확인일: 2026-10-14. 드라이버·타입 SQL. 문서 @rc 예시는 안정판 확정 전 복사 금지.

[S16] Vitest

https://vitest.dev/guide/

확인일: 2026-10-14. 단위·통합 테스트. MIT: https://github.com/vitest-dev/vitest

[S17] fast-check 모델 기반 테스트

https://fast-check.dev/docs/advanced/model-based-testing/

확인일: 2026-10-14. 명령 순서/상태 전이 성질 검증. MIT: https://github.com/dubzzz/fast-check

[S18] Playwright 접근성 테스트

https://playwright.dev/docs/accessibility-testing

확인일: 2026-10-14. 실브라우저/a11y 테스트. Apache-2.0: https://github.com/microsoft/playwright

[S19] MSW

https://mswjs.io/

확인일: 2026-10-14. 브라우저·Node 요청 mock. MIT: https://github.com/mswjs/msw

[S20] SQLite WAL

https://sqlite.org/wal.html

확인일: 2026-10-14. WAL 작동·제약. SQLite는 public domain.

[S21] SQLite transaction

https://sqlite.org/lang_transaction.html

확인일: 2026-10-14. 원자적 명령·이벤트·투영 갱신의 저장 기반.

[S22] SQLite foreign keys

https://sqlite.org/foreignkeys.html

확인일: 2026-10-14. 연결 제약 활성화·검증.

[S23] better-sqlite3

https://github.com/WiseLibs/better-sqlite3

확인일: 2026-10-14. MIT, 동기 SQL driver. native binary 호환성은 P0 검증.

[S24] OWASP SSRF Prevention

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

확인일: 2026-10-14. 외부 URL·리다이렉트·내부 주소 접근 방어 근거.

[S25] Brave Search API

https://api-dashboard.search.brave.com/app/documentation/web-search/get-started

확인일: 2026-10-14. 선택적인 실검색 Provider. 유료/정책 의존 SaaS이며 OSS로 분류하지 않음.

[S26] Node release schedule

https://nodejs.org/en/about/previous-releases

확인일: 2026-10-14. Node24 LTS를 기본 런타임 후보로 선택.

[S27] React versions

https://react.dev/versions

확인일: 2026-10-14. 확인 페이지 React19.2. 정확한 patch는 잠금 단계에서 선택.

[S28] Vite 공식 블로그

https://vite.dev/blog

확인일: 2026-10-14. 8.1 발표 2026-06-23 확인. 최신 patch 보장 아님.

[S29] pnpm workspaces

https://pnpm.io/workspaces

확인일: 2026-10-14. 내부 모노레포 패키지 연결.

[S30] Tailwind Vite 설치

https://tailwindcss.com/docs/installation/using-vite

확인일: 2026-10-14. 스타일 토큰·유틸리티 기반.

[S31] Storybook 테스트

https://storybook.js.org/docs/writing-tests

확인일: 2026-10-14. 상태별 컴포넌트 검증. Chromatic 유료 서비스는 필수 아님.

[S32] Biome

https://biomejs.dev/

확인일: 2026-10-14. 포맷·정적 품질 도구. lock 단계에 정확한 패키지 라이선스 확인.

[S33] GitHub Spec Kit

https://github.github.io/spec-kit/

확인일: 2026-10-14. 선택적인 SDD 문서 작업 도우미. Corral 명세/테스트를 대체하지 않음.

[S34] Drizzle 공식 저장소

https://github.com/drizzle-team/drizzle-orm

확인일: 2026-10-14. Apache-2.0. 안정판 태그·migration 호환성 검증.

[S35] xyflow 릴리스 조회

https://api.github.com/repos/xyflow/xyflow/releases?per_page=1

확인일: 2026-10-14. 2026-09-24 @xyflow/system@0.0.83 관측; React package version이 아님.

[S36] Pi 릴리스 조회

https://api.github.com/repos/earendil-works/pi/releases?per_page=1

확인일: 2026-10-14. 2026-10-12 v1.2.0 관측.

[S37] LangGraph.js 릴리스 조회

https://api.github.com/repos/langchain-ai/langgraphjs/releases?per_page=1

확인일: 2026-10-14. 2026-10-12 @langchain/vue@1.2.4 관측; runtime package version이 아님.

[S38] Fastify 릴리스 조회

https://api.github.com/repos/fastify/fastify/releases?per_page=1

확인일: 2026-10-14. 2026-10-13 v5.12.7 관측.

[S39] Vitest 릴리스 조회

https://api.github.com/repos/vitest-dev/vitest/releases?per_page=1

확인일: 2026-10-14. 2026-10-12 v5.0.4 관측.

[S40] LangGraph core license

https://github.com/langchain-ai/langgraphjs/blob/main/libs/langgraph-core/LICENSE

확인일: 2026-10-14. MIT, 해당 OSS 라이브러리 범위. 외부 호스팅 서비스 조건과 구별.

PART 08

문서 검증과 실행 준비

검증 범위 · 생성 파일 · 아직 검증하지 않은 부분

기준일: 2026-10-14.

문서·계약 검사: 513개 검사 통과.

요구 48개, 태스크 50개, 계획된 테스트 시나리오 112개를 연결했다. JSON Schema 문법, 예제 JSON, 일부 그래프 참조·포트, 태스크 의존 순환, 요구 추적, Markdown 링크·코드 블록을 검사했다.

Corral 제품 테스트를 112개 실행했다는 뜻이 아니다. 앱 구현, Pi/LangGraph 통합, 공급자 호출, 브라우저 E2E, 성능 시험은 실행하지 않았다. 해당 테스트들은 구현 플랜의 작업이다.

재검사: `python tools/validate_pack.py`. JSON 결과는 `tests/pack-validation-report.json`에 기록된다.

패키지 파일 구성

```
Corral-Spec-v0.1.md                제품·실행 스펙
Corral-Implementation-Plan-v0.1.md 구현 계획
README.md / AGENTS.md              안내·작업 규칙
docs/                              기술·API·ADR·출처·검증
contracts/                         요구·백로그·JSON Schema
tests/                             112개 계획 시나리오·추적·Gherkin
examples/                          정의·레이아웃·명령·검토 JSON
templates/                         기능 명세·변경 요청·PR
diagrams/                          Mermaid 원본 4종
tools/                             문서 팩 검증 스크립트
```